



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

SCHOOL OF MATHEMATICS

***Communication-Avoiding Krylov
Exponential Integration for Multi-GPU
Option Pricing***

KEVIN KNIGHTS

25378095

AUGUST 21, 2026

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
M.SC. IN HIGH-PERFORMANCE COMPUTING

(ACADEMIC YEAR 2025/26)

Declaration

I hereby declare that this summer project thesis is entirely my own work and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <http://www.tcd.ie/calendar>.

I have also completed the Online Tutorial on avoiding plagiarism 'Ready Steady Write', located at <https://libguides.tcd.ie/academic-integrity/ready-steady-write>.

I hereby certify that all AI tools used during the summer project and thesis writing have been detailed in section **AI**.

Signed:  _____

Date: August 21, 2026

Accompanying Git Repository

In the table below, commit and date refer to the last commit-id and commit-date before submitting this thesis. I am aware that all commits beyond the one mentioned here cannot be taken into account when evaluating the HPC summer project thesis at hand.

Owner: Kevin Knights
URL: https://github.com/kevinknights29/caksm
Commit: 0069442c81e5122ebab94c1fb86c6e599fa251a9
Date: 21 August 2026

Abstract

This thesis asks when two communication-avoiding (CA) mechanisms improve a Krylov exponential integrator for a three-asset Black–Scholes PDE. Matrix-powers tiling reduces data movement in sparse operator applications, while grouped orthogonalization reduces global reductions. Both mechanisms also add work, so they can lose when redundant arithmetic, halo exchanges, or local kernel costs outweigh the communication saved.

A dimensionless regime map organizes this comparison. Its vertical coordinate R_v compares the Arnoldi working set with last-level-cache capacity, and its horizontal coordinate R_h compares reduction latency with useful computation between reductions. Both are computed before timing from the problem and independently calibrated machine constants. Their unit lines identify opportunities, not universal crossovers. Cache geometry, access efficiency, overlap, and topology still determine whether an implementation recovers the potential saving.

Neither mechanism pays on the tested CPU. An 11–16 $\times$ reduction-count decrease gives no wall-clock improvement at $R_h = 0.080$, and halo work and access geometry keep cache-blocked matrix powers below their traffic ceiling. On a V100, a plane-streamed schedule cuts executed instructions by 2.88–3.45 $\times$. A measured runtime rule retains full volume for $n < 61$ and selects plane streaming from $n = 61$ when shared memory permits. It improves the complete solver by 1.248 $\times$ geometrically on four selection cases and 1.239 $\times$ on four active held-out cases, with worst cases of 1.204 $\times$ and 1.182 $\times$; four fallback controls remain near parity.

An equal-work distributed control removes an unused recurrence application from the $s = 1$ path. The resulting algorithm-relative ratio is 0.98–1.01 $\times$ on one V100 and 1.78–1.80 $\times$ on four V100s across two nodes. The gain is topology-specific: four V100s deliver only 0.89 $\times$ the one-GPU performance at fixed width, and the best four-GPU configuration remains 12–15% slower than the best one-GPU configuration.

The main contribution is a controlled framework for deciding when communication avoidance offers an opportunity and explaining why an implementation does or does not recover it. Independent Johnson and randomized quasi-Monte Carlo references establish total-price and Greek error, while measured orthogonality and an independent exponential-action referee provide the numerical acceptance evidence. The pricing application is used as a demanding case study; it is not presented as a state-of-the-art pricer.

Keywords: Krylov subspace methods, communication avoidance, matrix exponential, option pricing, performance modeling

Acknowledgements

I would like to thank my supervisor, Professor Kirk M. Soodhalter, for his support, guidance, and thoughtful feedback throughout the master's program. His teaching encouraged me to pursue research in numerical linear algebra, particularly Krylov subspace methods. As someone with an engineering background, I initially found the subject distant; his lectures, careful questions, and generosity with his time made it increasingly accessible. Our many discussions shaped this project and helped me grow as a researcher.

I am deeply grateful to my mother, Keyla, for her unconditional support despite the distance between us. Her love and encouragement sustained me through the difficult parts of this work.

I would also like to thank Jose Refojo for his patient help with modules, libraries, and configuration on the TCD HPC clusters. His support made the two-node, four-V100 experiments on Synge possible, and I am grateful for what I learned from him about CUDA and GPU computing.

I thank Richard Morrin and Niall O'Sullivan for fostering my interest in financial mathematics and computational finance. Their practical perspective, guidance, and assignments in C, C++, and quantitative finance gave me the foundation to pursue this application.

I am grateful to my classmates Edward, Louis, Eryk, and Conor for their support, friendship, and willingness to exchange ideas. I learned a great deal from them.

I am also grateful to the professors, staff, and students of the MSc in High-Performance Computing for the discussions and practical support that helped bring this work together.

Contents

Declaration	ii
Abstract	iii
Acknowledgements	iv
1 Introduction	1
2 The pricing problem and its discretization	2
3 The Krylov exponential integrator	4
4 Experimental platform and methodology	8
5 Financial validation	9
6 The OpenMP scaling study	15
7 Regime analysis	18
8 Recomputing the map for a GPU	29
9 The distributed CA exponential integrator	33
10 Conclusion	44
A Supporting studies and future work	45
AI Detailed statement about usage of AI tools	54

1 Introduction

A trading desk may solve the same multi-asset PDE many times while repricing a book. The wall-clock cost of one solve therefore affects the throughput available for valuation and risk analysis. Niesen and Wright’s option-pricing working paper reported an approximately 700 s historical CPU result for one three-asset configuration.¹ This figure provides historical context, but it is not a contemporary performance baseline.

In the implementation studied here, a Krylov exponential integrator spends over 70% of its time in sparse matrix–vector products (SpMV), which move far more bytes than they compute. Most of the remaining time is spent in an orthogonalization whose inner products require reductions. The first cost is data movement through the memory hierarchy; the second is synchronization between workers [1, 2]. Communication-avoiding (CA) methods target both costs. A matrix-powers kernel reuses an operator tile across several Krylov vectors [3, 4], while grouped orthogonalization reduces the number of synchronization phases [2, 5]. These savings come with extra arithmetic and storage.

Whether this exchange pays depends on the operator, grid, implementation, and machine. This thesis first identifies where communication avoidance has an opportunity to help and then tests whether the implementation realizes that opportunity on the available hardware.

Three artifacts support that study. A **benchmark pricer** in C++23 implements five methods (Crank–Nicolson, two ADI variants, a one-shot matrix exponential, and a Krylov exponential integrator) on two three-asset European options and isolates temporal error for a fixed semi-discretization (Section A.1). A **regime study** (Section 7) builds a dimensionless map of the CA design space. Its axes normalize demand by machine-dependent supply; their unit lines are opportunity indicators rather than promises of speedup or hardware portability. A **distributed CA exponential integrator** (Section 9) then evaluates those mechanisms on up to four GPUs across two nodes and asks whether their savings survive contact with a complete adaptive solver.

One design rule keeps the map non-circular: its coordinates are computed before a run from the problem size, sparsity pattern, spectrum, and independently measured machine constants. Wall-clock time, orthogonality loss, and achieved traffic do not enter the calculation. The experiment can therefore test the coordinates as opportunity indicators and leave the measured crossover as a result.

The thesis proceeds as follows. Section 2 states the pricing problem and its discretization. Section 3 presents the Krylov exponential integrator, its cost structure, and the CA reformulations that target it. Section 4 describes the experimental platform and measurement methodology. Section A.1 follows the historical Niesen–Wright option-pricing experiment as a temporal-

¹J. Niesen and W. M. Wright, *A Krylov Subspace Method for Option Pricing*, working paper, DOI: [10.2139/ssrn.1799124](https://doi.org/10.2139/ssrn.1799124).

integration control. Section A.3 profiles the solver and places its kernels on a cache-aware roofline. Section 6 presents the OpenMP scaling study that motivates the two CA mechanisms. Section 7 develops the regime map and its numerical and measured results. Section 8 recomputes the coordinates for a GPU memory system and tests their transfer across two distinct GPU architectures. Section 9 builds the distributed integrator and measures topology-specific performance on a full solve. Section A.4 names what the measurements leave open, and Section 10 concludes.

2 The pricing problem and its discretization

The performance study begins with the 3-asset Black–Scholes PDE [6] in log-price coordinates, solved backward in pseudo-time τ from 0 (payoff) to T (price):

$$\frac{\partial u}{\partial \tau} = \frac{1}{2} \sum_{d,d'} \rho_{dd'} \sigma_d \sigma_{d'} \frac{\partial^2 u}{\partial x_d \partial x_{d'}} + \sum_d \left(r - \frac{1}{2} \sigma_d^2 \right) \frac{\partial u}{\partial x_d} - r u + b(\tau). \quad (1)$$

The three groups of terms recur throughout the thesis under their standard PDE names. The second-derivative terms are *diffusion* (volatility spreading value across the surface; the mixed $d \neq d'$ terms couple correlated assets). The first-derivative terms are *convection*, also called *advection* (the risk-neutral drift transporting value across the grid). The zeroth-order term $-ru$ is *reaction* (here, discounting at the risk-free rate).

Two payoffs are priced. The basket call,

$$u_0(x) = \max \left(\sum_d w_d e^{x_d} - K, 0 \right),$$

and the rainbow min-call,

$$u_0(x) = \max \left(\min_d e^{x_d} - K, 0 \right).$$

The log-price domain is unbounded, so the finite computational box needs an artificial boundary closure. The closure follows the known far-field financial behavior of each payoff to limit truncation artifacts. For the basket call, nodes beyond the upper boundary are deep in the money: the basket value is far above the strike and the terminal payoff is overwhelmingly likely to remain in the exercise region. The option value there is well approximated by the basket value minus the discounted strike. Expanding $e^{-r\tau}$ to second order in τ and substituting this form into the ghost-node stencil gives the polynomial forcing $b(\tau) = \mathbf{Bs}(\tau)$, where $\mathbf{s}(\tau) = [\tau^2/2, \tau, 1]^\top$. This form reflects the payoff asymptote and allows the exact autonomous $N + 3$ augmentation used in Section 3. The rainbow case uses a zero-gamma closure, which locally continues the option value as a linear function. It is implemented through modified boundary stencils and requires no forcing augmentation.

Second-order central differences discretize the PDE on a uniform Cartesian grid. The code uses x_1 -fastest lexicographic ordering: grid points are numbered along x_1 first, then x_2 , and finally x_3 . Each one-dimensional operator then extends to the full grid as a three-fold Kronecker product with identity matrices in the undifferentiated directions. The assembled $\mathbf{A}$ has at most 19 nonzeros per row: a 7-point stencil and 3×4 mixed-derivative diagonals. The x_3 and mixed terms give it $O(N_1 N_2)$ bandwidth, which rules out banded direct solvers at scale.

The matrix splits as $\mathbf{A} = \mathbf{A}_{\text{sym}} + \mathbf{A}_{\text{skew}}$. Diffusion, mixed, and reaction terms are symmetric, while convection is skew-symmetric, so $\mathbf{A}$ is not symmetric in general. Its measured eigenvalues have nonpositive real parts, consistent with asymptotic decay. However, the spectrum alone does not control transient amplification for a non-normal matrix. Stability and approximation quality are therefore checked with exponential-action and residual diagnostics instead of being inferred from eigenvalues alone [7]. Table 1 lists the initial model parameters. The financial-validation sweep later widens the domain from $\alpha = 2.85$ to the accepted $\alpha = 4.275$ while keeping the central spacing approximately fixed (Section 5.1).

Table 1: Default model parameters.

Parameter	Value	Description
Spot prices S_0	(100, 100, 100)	Initial asset prices
Strike K	100	Option strike
Risk-free rate r	0.04	Continuously compounded
Maturity T	1.0 year	Fixed
Volatilities σ	(0.30, 0.35, 0.40)	Per-asset annual volatility
Correlations ρ	(0.50, 0.50, 0.50)	Off-diagonal pairs ($\rho_{01}, \rho_{02}, \rho_{12}$)
Basket weights w	(1/3, 1/3, 1/3)	Equal-weight basket
Grid half-width α	2.85	Initial domain $\pm \alpha \sigma \sqrt{T}$ per dimension

Stiffness grows quadratically with refinement because the central second difference of the diffusion term on n interior points per axis has eigenvalues

$$\lambda_k = -\frac{2\sigma^2}{\Delta x^2} \left(1 - \cos \frac{k\pi}{n+1} \right), \quad k = 1, \dots, n,$$

Thus the extreme eigenvalue scales as $O(\Delta x^{-2}) = O(n^2)$ [7]. The uniform box has axis-dependent spacing $\Delta x_d = 2\alpha\sigma_d\sqrt{T}/(n-1)$. The spectrum quoted here is measured from the fully assembled three-dimensional operator with those three spacings. Its most negative eigenvalue moves from -24 at $n = 11$ to -2924 at $n = 101$, where $N = n^3$, which is consistent with the predicted scaling. Once temporal error falls below the spatial-discretization and domain-truncation errors of a fixed grid, smaller time steps cannot improve the total option-price error. Further temporal work would only over-solve that grid.

3 The Krylov exponential integrator

Spatial discretization turns the pricing PDE into a large linear ODE. A system is *autonomous* when its right-hand side depends on the state but not explicitly on time; it is *non-autonomous* when time appears explicitly. For the basket payoff, the time-dependent boundary forcing makes this system non-autonomous:

$$\frac{d\mathbf{u}}{d\tau} = \mathbf{A}\mathbf{u} + \mathbf{B}\mathbf{s}(\tau), \quad \mathbf{s}(\tau) = \begin{bmatrix} \tau^2/2 & \tau & 1 \end{bmatrix}^\top, \quad \mathbf{u}(\tau_0) = \mathbf{u}_0. \quad (2)$$

The three-column matrix $\mathbf{B}$ contains the coefficients of the polynomial boundary forcing described in Section 2. Since $\mathbf{s}'(\tau) = \mathbf{K}\mathbf{s}(\tau)$ for the nilpotent shift

$$\mathbf{K} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix},$$

the forcing can be absorbed exactly into an autonomous augmented system. Define

$$\tilde{\mathbf{A}} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{K} \end{bmatrix} \in \mathbb{R}^{M \times M}, \quad M = N + 3, \quad \tilde{\mathbf{v}} = \begin{bmatrix} \mathbf{u}_k \\ \mathbf{s}(\tau_k) \end{bmatrix} \in \mathbb{R}^M. \quad (3)$$

Then the first N components of $e^{h\tilde{\mathbf{A}}}\tilde{\mathbf{v}}$ equal $\mathbf{u}(\tau_k + h)$, while the last three components equal $\mathbf{s}(\tau_k + h)$. The rainbow payoff has no forcing matrix in the implementation. For that case $M = N$, $\tilde{\mathbf{A}} = \mathbf{A}$, and $\tilde{\mathbf{v}} = \mathbf{u}_k$.

For one step, define the scaled operator $\mathbf{G} = h\tilde{\mathbf{A}}$. The implementation applies Arnoldi to $\mathbf{G}$ with starting vector $\tilde{\mathbf{v}}$, producing an orthonormal basis $\mathbf{V}_m$ and an upper Hessenberg matrix $\bar{\mathbf{H}}_m \in \mathbb{R}^{(m+1) \times m}$ [8, 9]. Dropping the last row gives $\mathbf{H}_m \in \mathbb{R}^{m \times m}$, and the approximation reads

$$e^{h\tilde{\mathbf{A}}}\tilde{\mathbf{v}} \approx \beta \mathbf{V}_m e^{\mathbf{H}_m} \mathbf{e}_1, \quad \beta = \|\tilde{\mathbf{v}}\|. \quad (4)$$

Algorithm 1 gives the adaptive procedure; its Arnoldi iteration is Algorithm 2.

The scalar η_m in Algorithm 1 has a specific, limited meaning. Arnoldi is applied to the scaled operator $h\tilde{\mathbf{A}}$. For the projected trajectory $\tilde{\mathbf{y}}_m(\theta) = \beta \mathbf{V}_m e^{\theta \mathbf{H}_m} \mathbf{e}_1$, $0 \leq \theta \leq 1$, the endpoint defect with respect to the scaled coordinate θ is parallel to $\mathbf{v}_{m+1}$ and has norm

$$\eta_m = \beta |\bar{h}_{m+1,m}| |\mathbf{e}_m^\top e^{\mathbf{H}_m} \mathbf{e}_1|.$$

The implementation uses 10^{-8} as a fixed double-precision engineering tolerance for every Krylov action. This value was selected before the experiments. It keeps the required Krylov dimension moderate and is stricter than the temporal accuracies compared in Section A.1, but it is not a calibrated total-price tolerance. Without a separate stability estimate or an integrated

Algorithm 1: Adaptive Krylov Exponential Integrator for the Basket Polynomial Forcing

Input: Matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, forcing coefficients $\mathbf{B} \in \mathbb{R}^{N \times 3}$, initial condition $\mathbf{u}_0$, final time T , time-step $\Delta\tau$, absolute defect tolerance tol , maximum dimension $m_{\max}$

Output: Solution snapshots $\{\mathbf{u}_k\}_{k=0}^K$ at $\tau_k = k \Delta\tau$

```

1  $\mathbf{K} \leftarrow \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}; \quad \tilde{\mathbf{A}} \leftarrow \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{K} \end{bmatrix}; \quad M \leftarrow N + 3;$ 
2  $\tau \leftarrow 0;$ 
3 for  $k = 0, 1, \dots, K - 1$  do
4    $h \leftarrow \min(\Delta\tau, T - \tau);$ 
5    $\mathbf{G} \leftarrow h\tilde{\mathbf{A}};$ 
6    $\tilde{\mathbf{v}} \leftarrow [\mathbf{u}_k^T, \mathbf{s}(\tau)^T]^T$  // Eq. (3)
7    $m \leftarrow 1; \quad \eta_m \leftarrow \infty;$ 
8   /* Adaptive Krylov loop */
9   while  $m \leq \min(M, m_{\max})$  do
10    Run  $m$  Arnoldi iterations on  $(\mathbf{G}, \tilde{\mathbf{v}}) \Rightarrow \mathbf{V}_{m+1}, \bar{\mathbf{H}}_m$  // Algorithm 2
11    Store  $\bar{h}_{m+1,m} \leftarrow [\bar{\mathbf{H}}_m]_{m+1,m}$  // sub-diagonal entry
12     $\mathbf{H}_m \leftarrow \bar{\mathbf{H}}_m(1:m, 1:m)$  // drop last row
13     $\beta \leftarrow \|\tilde{\mathbf{v}}\|;$ 
14     $\mathbf{f} \leftarrow e^{\mathbf{H}_m} \mathbf{e}_1$  // small  $m \times m$  matrix exponential
15     $\mathbf{w} \leftarrow \beta \mathbf{V}_m \mathbf{f}$  // Krylov approximation in full space
16    /* Endpoint residual/defect indicator */
17     $\eta_m \leftarrow \beta |\bar{h}_{m+1,m}| \|\mathbf{f}\|_m;$ 
18    if  $\eta_m < \text{tol}$  then
19      break;
20    end
21     $m \leftarrow m + 1;$ 
22  end
23   $\mathbf{u}_{k+1} \leftarrow \mathbf{w}(1:N)$  // extract first  $N$  components
24   $\tau \leftarrow \tau + h;$ 
25 end

```

Algorithm 2: Arnoldi Iteration for the Scaled Operator

Input: Scaled matrix $\mathbf{G} = h\tilde{\mathbf{A}} \in \mathbb{R}^{M \times M}$, vector $\tilde{\mathbf{v}} \in \mathbb{R}^M$, subspace dimension m , breakdown tolerance $\text{tol}_{\text{break}}$

Output: Orthonormal basis $\mathbf{V}_{m+1} \in \mathbb{R}^{M \times (m+1)}$, upper Hessenberg $\bar{\mathbf{H}}_m \in \mathbb{R}^{(m+1) \times m}$

```

1  $\beta \leftarrow \|\tilde{\mathbf{v}}\|; \quad \mathbf{v}_1 \leftarrow \tilde{\mathbf{v}}/\beta;$ 
2 for  $j = 1, \dots, m$  do
3    $\mathbf{w} \leftarrow \mathbf{G} \mathbf{v}_j;$ 
4   for  $i = 1, \dots, j$  do
5     // Modified Gram-Schmidt
6      $h_{i,j} \leftarrow \mathbf{v}_i^T \mathbf{w};$ 
7      $\mathbf{w} \leftarrow \mathbf{w} - h_{i,j} \mathbf{v}_i;$ 
8   end
9    $h_{j+1,j} \leftarrow \|\mathbf{w}\|;$ 
10  if  $h_{j+1,j} < \text{tol}_{\text{break}}$  then
11    break // Invariant subspace found (happy breakdown)
12  end
13   $\mathbf{v}_{j+1} \leftarrow \mathbf{w}/h_{j+1,j};$ 
14 end

```

defect, the indicator does not bound the state error, global time-integration error, or option-price error. An independent exponential-action referee supplies the solution-accuracy check used later in the thesis.

GPU shared memory defines the production cap. The projected exponential uses four static $m \times m$ binary64 work arrays and two shared integers. Both tested architectures allow at most 48 KiB of statically allocated shared memory per CUDA thread block. Larger opt-in allocations apply to dynamic shared memory and would require a kernel redesign [10]. The largest common value for the unchanged kernel is therefore $m_{\max} = 39$. It uses 48,680 bytes; $m = 40$ would require 51,208 bytes. The CMake cache variable `CAKSM_GPU_CA_MAX_M` makes the cap configurable for other builds and rejects values above the static limit.

The dense calibration uses the actual $n = 5$ Basket and Rainbow discretizations. This grid keeps the three-dimensional stencil and both boundary treatments while making a dense matrix exponential inexpensive. The sweep covers $m = 1, \dots, 39$ at the first and midpoint time steps for $T = 0.5$ and $h = 0.005$. All 40 stopping cases meet their requested tolerance and select at most $m = 6$. The hardware ceiling therefore does not determine their numerical stopping point.

Unless a reduced-precision experiment is named explicitly, every numerical kernel in this thesis uses IEEE 754 binary64 (C++ double). For calibration, the *true action error* is the norm of the difference between the Krylov approximation and the dense value $\exp(h\tilde{\mathbf{A}})\tilde{\mathbf{v}}$ for one time step. Once that difference reaches the dense reference’s rounding floor, it no longer resolves decreasing truncation error. Before the floor, η_m is 1.004 to 5.006 times the true action error. At tolerance 10^{-8} it selects $m = 5$, where the action error is 3.8×10^{-12} to 1.5×10^{-11} ; $s = 1$ and $s = 4$ agree at the shown precision. Across the full $m \leq 39$ sweep, the relative Arnoldi-residual stays below 8.2×10^{-14} and orthogonality loss below 2.0×10^{-13} . For these small actions, the stopping indicator is conservative. This result is not a universal bound and does not guarantee that $m_{\max} = 39$ is sufficient on larger grids.

3.1 Cost structure and the sequential dependency

After numerical acceptance, the next question is cost. Arnoldi dominates. Each iteration j requires one sparse matrix–vector product $\mathbf{w} = \tilde{\mathbf{A}}\mathbf{v}_j$, costing $O(N)$, and orthogonalization against all previous vectors, costing $O(jN)$. Iteration j cannot start until iteration $j - 1$ produces $\mathbf{v}_j$, creating a sequential dependency across the Krylov dimension m . The SpMV and inner products within one iteration can be parallelized, but they scale poorly for different reasons: SpMV is bandwidth-bound, while every inner product in the modified Gram–Schmidt (MGS) loop requires a global reduction. These are the two *communication* walls in Table 2.

Table 2: The two kernels, their bounds, and the CA mechanism each motivates.

Kernel	Bound	Communication	CA mechanism
SpMV	bandwidth (memory hierarchy)	vertical	matrix powers / cache blocking
Gram–Schmidt	synchronization (reductions)	horizontal	s -step / Krylov-column orthogonalization

3.2 Communication-avoiding reformulations

The two communication walls motivate two CA reformulations and one terminology convention. A q -column *Krylov block* is $\mathbf{Z}_q = [\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^{q-1}\mathbf{v}]$ and needs $q - 1$ operator applications; it is neither a partition of $\mathbf{A}$ nor a CUDA execution unit. A *CUDA thread block* is a scheduled thread group, a *spatial tile* the grid region it stages, and a *slab* one GPU’s subdomain. The OpenMP label block in Section 6.3 instead names a row schedule. A *time step* advances τ , an *Arnoldi iteration* adds one Krylov vector, and a *recurrence application* multiplies by $\mathbf{A}$ within a Krylov block. Archived CPU degrees are converted by $q_{\max} = s_{\max} + 1$; distributed tables retain the executable’s $s = q$.

The CA-Arnoldi reformulation [2, 3, 5] constructs one such Krylov block per communication phase. The *matrix-powers kernel* [4] reuses an operator tile across the $q - 1$ applications, attacking the vertical wall. The Krylov block is then orthogonalized at once—here by CholeskyQR2 [11]—which reduces the number of synchronization phases relative to MGS.

The condition-number test used by the implementation requires a qualification that is important for every result below. For a tall matrix $\mathbf{Z} \in \mathbb{R}^{M \times q}$, the cited CholeskyQR2 analysis includes the dimension-dependent sufficient condition

$$\delta = 8 \kappa_2(\mathbf{Z}) \sqrt{Mqu + q(q+1)u} \leq 1, \quad (5)$$

in addition to its other hypotheses. The simpler line $\kappa_2(\mathbf{Z}) \leq u^{-1/2}$ describes a practical breakdown scale for the normal equations; it is not a dimension-free sufficient theorem. For the production dimension, $M = 61^3 + 3 = 226,984$ and $q = 4$, so (5) limits $\kappa_2(\mathbf{Z})$ to approximately 1.25×10^4 . The value 6.0×10^6 is the largest condition number recorded over the evolving width-four monomial Krylov blocks in the $n = 61$ Rainbow integration reported in Table 27. Substitution gives $\delta \approx 482$. Because (5) is sufficient rather than necessary, this value makes the theorem inconclusive for the measured Krylov block; it does not diagnose instability. The distributed Gram matrix also introduces all-reduce rounding beyond the serial formulation.

Throughout the thesis, $u^{-1/2}$ is therefore called the *implemented conditioning guard* or *heuristic line*. Its operational role is to detect rapidly growing Krylov-block conditioning and trigger a narrower Krylov block before normal-equations breakdown. Numerical acceptance is based

on the guard together with Cholesky success, measured orthogonality, and final-state agreement with an independent referee. This separates the theorem’s sufficient guarantee from the direct evidence available for the tested implementation. The regime study then measures how the monomial Krylov-block conditioning grows with q and how that growth affects the width selected by the guard.

Rational Krylov offers an alternative by replacing the sequential polynomial recurrence with independent shifted solves. Berljafa and Güttel analyze a parallel rational Arnoldi implementation, including simultaneous shifted solves and their numerical conditioning [12]. The parallelism is attractive, but each shifted solve is a sparse linear system whose iterative solution reintroduces both communication walls. Near-optimal pole selection also adds problem-specific tuning. This study uses the s -step route because its costs can be expressed directly in the machine terms of the regime map.

Randomized orthogonalization is another promising direction. It constructs a well-conditioned basis whose much smaller random sketch is orthonormal, which can reduce arithmetic and communication while retaining a randomized Arnoldi relation [13]. It is not ranked against CA-Arnoldi here because it was not implemented or measured on this pricing operator. A useful follow-up would apply the same regime coordinates and test whether the lower synchronization cost improves complete-solver time on these machines.

4 Experimental platform and methodology

The regime coordinates are meaningful only if the cache, memory, and communication hierarchies can be measured directly. Every CPU result in this thesis was collected on one bare-metal workstation (“puffin”), an AMD Ryzen Threadripper 3960X with 24 cores in a single socket and NUMA node. Its 8 *core complexes* (CCX) each contain 3 cores that share one 16 MiB L3 slice, giving 128 MiB in total. Each core has a 32 KiB L1d cache, a 512 KiB L2 cache, a nominal maximum clock of 3.8 GHz, and a second hardware thread through simultaneous multithreading (SMT). Main memory is 4-channel DDR4-3200 with a theoretical 102.4 GB/s roof. Puffin also hosts an NVIDIA RTX 3090 with 24 GB, one half of the GPU comparison below. Its derived constants are centralized in `include/machine.hpp` under the key `amd-3960x`; the regime study computes every predicted CPU coordinate from them.

The GPU platform. The GPU study of Section 8 adds a second machine, “syngé,” a two-node cluster carrying a pair of NVIDIA Tesla V100-PCIE-16GB cards per node (four in total). The two cards within a node communicate through PCIe across a cross-socket hop with no NVLink, and the nodes use a 100 Gb/s Omni-Path fabric. The NVIDIA Collective Communications Library (NCCL) provides GPU-aware collective and point-to-point operations; on Syngé it selects an InfiniBand-verbs transport over that fabric. Syngé’s V100 and puffin’s RTX 3090

form a useful but not controlled architectural contrast. Their measured bandwidths and nominal L2 capacities are similar, while their FP64 peaks differ by about $11\times$; they also differ in SM organization, cache hierarchy, clocks, memory technology, ECC behavior, schedulers, and launch behavior. Conclusions from the pair are therefore kernel- and architecture-specific rather than causal estimates of FP64 throughput alone. Their constants live in a separate header (`include/gpu_machine.hpp`), typed distinctly from the CPU constants so that a CPU value reaching a GPU coordinate is a compile error rather than a plausible wrong number, and are calibrated per card on an idle device rather than read from datasheets.

Workstation selection. A cloud arm was considered because cloud providers offer server-grade hardware at reasonable hourly cost, larger problem configurations, and an environment relevant to commercial research. It was omitted because co-tenancy can produce timing variation that cannot be separated from genuine memory-bound behavior inside the guest. Virtual topology is a larger limitation because the hypervisor reports a socket and private last-level-cache slice per virtual CPU rather than the physical cache hierarchy. The regime coordinates depend on that hierarchy through cache capacity and the boundaries crossed by a reduction tree, so they cannot be evaluated reliably on the reported topology. The study therefore requires an observable physical topology and explicit contention controls. Idle-device gating supplies the corresponding safeguard on the shared cluster. Bare metal removes the hypervisor but not operating-system scheduling noise, so timed quantities are reported as medians with the stated spread over repeated trials. On a boost-enabled processor, the median represents a typical repeat, whereas the minimum can reflect a transient boost state.

Reproducibility. Each reported result can be traced from the implementation to its figure. The repository contains the CMake build, C++23 experiment programs, centralized machine descriptions, run scripts, and a named plotting script for each figure with inline dependency metadata. Tables and captions record the relevant parameters and timing protocols, and the executables use the same audited constants that define the regime coordinates. Additional run metadata, raw measurements, and profiler outputs are available from the author on request.

5 Financial validation

The references in Section A.2 make it possible to ask a total-error question: how far is the computed price from an independent financial price? The experiments refine domain width, grid spacing, and temporal resolution in that order so that each comparison isolates one source of error.

5.1 The working domain

Domain truncation is tested first because grid refinement cannot remove it. Each domain in Table 3 rescales the base pair $(\alpha_0, n_0) = (4.275, 61)$ to an exactly odd grid size. This keeps the central spacing matched to roundoff: the largest relative mismatch across the sweep is 4.9×10^{-16} . A change in spacing therefore cannot be mistaken for a domain effect.

At the repository's historical $\alpha = 2.85$ the Basket price is 2.0×10^{-2} below the reference, and about 1.6×10^{-2} of that is domain truncation, which grid refinement cannot remove. At $\alpha = 4.275$ the remaining domain effect, measured between the two widest domains, is 1.0×10^{-4} for the Basket and 4.5×10^{-6} for the Rainbow, both an order of magnitude below the accompanying spatial error. This is the accepted domain for the validation experiments that follow.

The Rainbow row at $\alpha = 2.1375$ is a useful warning about *cancellation error*. Its price is closer to the reference than at any wider domain only because the domain and spatial errors have opposite signs. The narrow domain is not more accurate, and choosing α from the smallest observed total error would select the wrong one.

Table 3: Domain truncation at matched central spacing, smoothed HV-ADI at 200 time steps. Each grid size is the base pair rescaled, so Δx is held fixed to roundoff across the sweep.

α	n	Basket		Rainbow	
		Price	Error	Price	Error
2.1375	31	13.136441	-1.08×10^{-1}	4.478207	$+3.32 \times 10^{-2}$
2.85	41	13.224686	-2.02×10^{-2}	4.401538	-4.35×10^{-2}
3.5625	51	13.239449	-5.45×10^{-3}	4.397490	-4.75×10^{-2}
4.275	61	13.241017	-3.88×10^{-3}	4.397329	-4.77×10^{-2}
4.9875	71	13.241120	-3.78×10^{-3}	4.397325	-4.77×10^{-2}
Remaining domain effect		1.03×10^{-4}		4.50×10^{-6}	

5.2 Spatial and temporal refinement

At the accepted domain, both payoffs converge at second order and the two methods agree. Table 4 gives the base case. Figure 1 shows both trends and draws the reference uncertainty as a floor, making it clear when a plotted error would only report noise.

Prices approach the references from below, as Dang, Christara and Jackson also report [14]. Only the order and direction are compared: their linear domain and one-sided boundary scheme differ from the log-price grid and boundary closures used here, so the absolute error constants describe different discretizations.

After the start-up region, the temporal trend is second order for both payoffs. This includes the Basket with its time-dependent boundary forcing, which the two HV stages treat differently

and which a misplaced forcing term would affect first. Tightening the Krylov tolerance from 10^{-4} to 10^{-12} changes the Basket price by 5.0×10^{-9} and the Rainbow price by 2.4×10^{-9} . Both changes are four orders of magnitude below the spatial error, so Krylov temporal error is negligible in this grid's budget.

5.3 Delta and diagonal Gamma

Price convergence alone does not validate the extracted sensitivities. Their accuracy also depends on how derivatives are reconstructed from the discrete field. Both methods therefore extract price and Greeks from one solved field per configuration. Trilinear interpolation forms a weighted average of the eight corners of the surrounding cell and is linear in each log-price coordinate. Rebuilding a centered grid for every bumped spot would mix the finite-difference error with changes in the discretization, so every term is evaluated from the same field.

Relative to the independent references, the Basket Delta errors are 0.011–0.020%; the Rainbow Delta errors are 0.17–0.24%. These are small multiples of the corresponding relative price errors, as expected for a first derivative extracted from a second-order field.

The interpolated diagonal Gammas do not agree with the references, and the extraction method explains why. A trilinear interpolant is linear inside a cell, so its second difference collapses when the bump is smaller than the grid spacing. The estimate then becomes the grid's curvature scaled by the ratio of spacing to bump. Even a two percent spot bump spans less than one cell on the accepted grid. The departure follows this span. For the Rainbow, the relative excess falls from 0.21 at 0.69 cells to 0.12 at 0.79 and 0.023 at 0.93; for the Basket, it falls from 3.4 at 0.40 cells to 0.15 at 0.93, approximately as the reciprocal of the span.

The grid-node stencil confirms that extraction is the source. Its Basket centers differ by 0.011–0.066%, which lies inside their stated 0.05–0.22% randomized-QMC uncertainty. They are consistent, not separately resolved. The Rainbow differences are 0.54–0.88% against 0.13–0.30% uncertainty. The stencil therefore resolves curvature at the reported accuracy, while trilinear extraction loses it (Table 4). A sub-cell bump is unreliable, but a multi-cell bump introduces increasing $O(h^2)$ truncation error.

5.4 The equal-accuracy comparison

With total accuracy established, the next comparison asks how much each method costs at a common numerical target. Both methods run with one thread on the same host, domain, and grid, which isolates algorithmic work at the declared accuracy. GPU KSM-EI timings elsewhere in the thesis measure application throughput and are not included in the speedup ratios of Table 5.

At a fixed grid both methods converge to the same semi-discrete answer, so the remaining

question is how much temporal work each needs to reach a declared total price-error target. For each payoff the target is

$$\max(1.05 |V_{\text{grid}}^{\text{conv}} - V_{\text{ref}}|, 4 \delta_{\text{ref}}),$$

where $V_{\text{grid}}^{\text{conv}}$ is that grid's temporally converged price and δ_{ref} is the reference uncertainty. The first term gives the grid's unavoidable error a 5% margin; the second prevents claiming resolution below the reference. Here the first term binds, giving $1.05(3.884 \times 10^{-3}) = 4.08 \times 10^{-3}$ for the Basket and $1.05(4.772 \times 10^{-2}) = 5.01 \times 10^{-2}$ for the Rainbow.

Spatial and temporal errors are signed differences. A coarse time step can shift the price upward while the converged spatial solution lies below the reference. Their sum can then become smaller even though neither component improved. To avoid accepting this cancellation, the next finer temporal setting must also meet the target. Timings exclude shared PDE assembly and include method-specific setup; one warm-up is discarded and seven repetitions are measured.

5.5 The price and Greek error hierarchy

Table 6 separates the price-scale error sources so that the limiting term is visible for each payoff. No claim is carried to more significant digits than its largest unresolved component supports.

Table 4: Base case at the accepted domain $\alpha = 4.275$, $n = 121$, 200 time steps, Krylov tolerance 10^{-12} . Greeks are quoted at the bump the reference sweep accepted for each one. The two solvers agree to every digit shown on every Greek, so the informative comparison is between the interpolated and node-stencil extractions of the same field.

Payoff	Quantity	Bump	Reference	Uncert.	Interpolated	Node stencil
Basket	Price	—	13.244899	6.8×10^{-6}	13.243948	—
Basket	Δ_1	0.005	0.197035	2.4×10^{-6}	0.197056	0.197064
Basket	Δ_2	0.005	0.203353	2.5×10^{-6}	0.203384	0.203393
Basket	Δ_3	0.005	0.209812	3.1×10^{-6}	0.209853	0.209864
Basket	Γ_{11}	0.050	0.001587	7.4×10^{-7}	0.001730	0.001588
Basket	Γ_{22}	0.020	0.001551	3.4×10^{-6}	0.002435	0.001550
Basket	Γ_{33}	0.020	0.001497	2.4×10^{-6}	0.003027	0.001498
Rainbow	Price	—	4.445018	$< 10^{-13}$	4.433070	—
Rainbow	Δ_1	0.005	0.105830	7.4×10^{-6}	0.105573	0.105570
Rainbow	Δ_2	0.005	0.085719	4.7×10^{-6}	0.085556	0.085553
Rainbow	Δ_3	0.005	0.072605	7.3×10^{-6}	0.072485	0.072482
Rainbow	Γ_{11}	0.020	−0.001931	2.4×10^{-6}	−0.001976	−0.001916
Rainbow	Γ_{22}	0.010	−0.001757	4.9×10^{-6}	−0.003064	−0.001741
Rainbow	Γ_{33}	0.020	−0.001505	4.5×10^{-6}	−0.001825	−0.001497

The price rows carry the accepted errors of both solvers: -9.51×10^{-4} for the smoothed ADI

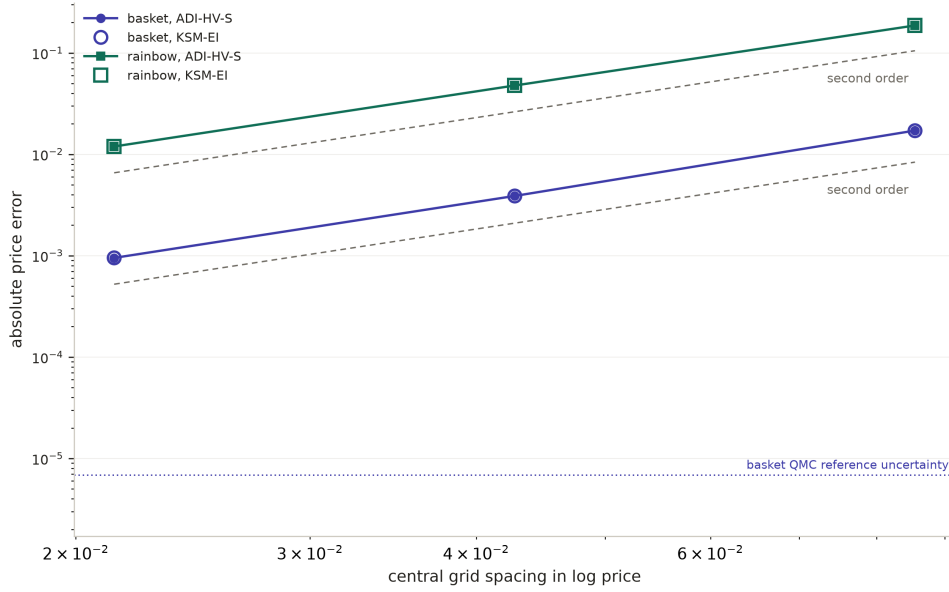


Figure 1: Absolute price error against central grid spacing at the accepted domain, for both payoffs and both methods, with second-order guides. The two methods agree to about 3×10^{-5} at every grid, two orders below the spatial error they share, so their curves coincide at this scale. The sampled Basket reference’s uncertainty is drawn as a floor; the analytic Rainbow reference is stable below 10^{-13} and its floor would lie ten decades below the plot.

and -9.54×10^{-4} for the Krylov integrator on the Basket, -1.19×10^{-2} and -1.20×10^{-2} on the Rainbow. Both are dominated by the spatial term, which the two methods share.

Table 5: Equal-accuracy comparison on puffin at $n = 61$, $\alpha = 4.275$, one thread, one discarded warm-up and seven measured repetitions. Timings exclude the shared PDE assembly and include all method-specific setup. A configuration is admissible only if its distance from the grid’s converged price fits inside what the target does not already spend on that grid’s spatial error, so neither row reaches its accuracy by cancellation.

Payoff	Method	Time steps	Price	Error	Median (s)	Range (s)
Basket	ADI-HV-S	50	13.241066	3.83×10^{-3}	3.090	3.076–3.095
Basket	KSM-EI	1	13.240901	4.00×10^{-3}	0.533	0.532–0.536
<i>Declared target</i> 4.08×10^{-3}				<i>reference floor</i> 6.8×10^{-6}		
Rainbow	ADI-HV-S	50	4.397854	4.72×10^{-2}	2.936	2.914–2.944
Rainbow	KSM-EI	1	4.397355	4.77×10^{-2}	0.460	0.458–0.462
<i>Declared target</i> 5.01×10^{-2}				<i>reference floor</i> $< 10^{-13}$		

The Krylov price is unchanged to thirteen digits from one to 400 time steps because this autonomous linear problem can be advanced by one exponential action to the stopping tolerance. Its cheapest admissible configuration therefore uses one time step. Under the same declared-error rule, the start-up-smoothed HV method needs 50 time steps. The resulting one-thread speedups are $5.80\times$ for the Basket and $6.39\times$ for the Rainbow, with non-overlapping timing ranges.

Table 6: Price-error hierarchy at the accepted working point. The largest unresolved component is the spatial discretization for both payoffs, so no price is claimed to more digits than it supports.

Component	Basket	Rainbow	Evidence
Independent reference	6.8×10^{-6}	$< 10^{-13}$	QMC half-width; Johnson quadrature stability
Domain truncation	1.0×10^{-4}	4.5×10^{-6}	two widest matched-spacing domains
Spatial discretization	2.9×10^{-3}	3.6×10^{-2}	odd-grid refinement at the accepted domain
ADI temporal	2.2×10^{-6}	2.4×10^{-5}	smoothed HV time-step refinement, finest pair
KSM temporal	5.3×10^{-15}	3.6×10^{-15}	tolerance refinement and the defect calibration

The hierarchy gives a clear next step. At the accepted working point, every term except spatial discretization is at least one order of magnitude smaller, and the Krylov temporal term is ten orders smaller. Improving either price requires grid refinement. The references are already at least two orders of magnitude more accurate than the current spatial result.

The separate Greek hierarchy is in Table 4: uncertainty combines QMC sampling and finite-difference truncation, node versus reference tests the spatial field, and interpolated versus node isolates extraction. Off-node Gamma is therefore not folded into the price-error table.

5.6 Scope of the validation

Section A.1 validates time integration against the accurate exponential action of the same spatial operator; Section 5 validates the complete discretized price against an independent financial reference. Consequently, a 10^{-11} semi-discrete difference can coexist with a 10^{-3} total price error: the latter is dominated by the grid, not the time integrator.

The Greek accuracy also depends on where it is extracted. Delta and node-stencil Gamma are limited principally by spatial discretization and agree with their references. An *off-node* Gamma is the second spot derivative requested at a location that is not a grid node, so it must first be reconstructed from nearby field values. Trilinear interpolation has no within-cell curvature and cannot support that derivative below one cell. Higher-order tensor-product B-spline finite-element interpolation has been shown to deliver high-order convergence for option prices, Delta, and Gamma [15]; adapting such an interpolant is the direct opportunity here. This thesis does not implement it and therefore makes no accuracy claim for off-node Gamma.

The scope is three European assets under one model. The references assume lognormal terminal distributions with fixed correlation, the Johnson formula is specific to a call on the minimum, and the domain and grid were selected for these two contracts at these parameters. The results do not cover high-dimensional baskets, path dependence, early exercise, or stochastic volatility.

The equal-accuracy table also compares two CPU methods inside this codebase; it is not a comparison of production pricing systems.

6 The OpenMP scaling study

The scaling study measures how the two hot kernels in one Arnoldi cycle respond to additional cores. Solution accuracy is not part of this experiment. The important distinction is that the kernels fail to scale for two *different* reasons, each motivating a different CA mechanism (Table 2). Section 7 later turns this split into two independent dimensionless axes.

6.1 Instrument design

The harness freezes the Krylov dimension at $m = 8$ and disables the convergence check so that both ceilings are visible. This value is the rounded mean dimension measured for the $n = 31$ solve ($\bar{m} = 7.60$ in Table 30) and lies inside the measured $m = 8$ –11 range used by the reduction ledger. A fixed m keeps the Gram–Schmidt/SpMV work ratio unchanged. Runtime variation across the core count P can then be attributed to hardware rather than a changing algorithm, while genuine $m(n)$ growth is measured separately.

The hand-written SpMV stores the matrix in compressed sparse row (CSR) format, with each row’s nonzeros held contiguously. Writing the kernel for this study keeps the row partition and reduction strategy under the experiment’s control. Each thread is the only writer of a contiguous, disjoint output slice, and partition seams are cache-line aligned. The harness runs MGS with parallel global reductions and reports per-kernel timings, so one sweep shows both SpMV’s plateau and Gram–Schmidt’s roll-off. Threads are pinned with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`. Each CCX (Section 4) fills before the next one is used, exposing per-CCX L3 bandwidth as discrete steps. Core counts align with CCX boundaries, and SMT lanes are excluded so that no timed core shares its pipelines with a second thread. Each point is the median of five to seven timed repeats after one warm-up, with the interquartile range as the error bar.

Two initialization arms form a controlled A/B experiment on memory placement. Arm A fills the matrix from thread 0. In arm B, each thread first-touches the row partition it will later compute. The arms are indistinguishable for the reason examined in Section 6.3. Figures 2 and 3 show the strong- and weak-scaling results.

6.2 The quarantined $m(n)$ growth

The harness freezes m so that the scaling curves measure hardware. The genuine growth of the Krylov dimension is measured separately (Figure 4). Its mean per time step rises from 7.6 at $n = 31$ to 11.3 at $n = 89$. This growth is slow, but it is enough to increase Gram–Schmidt’s

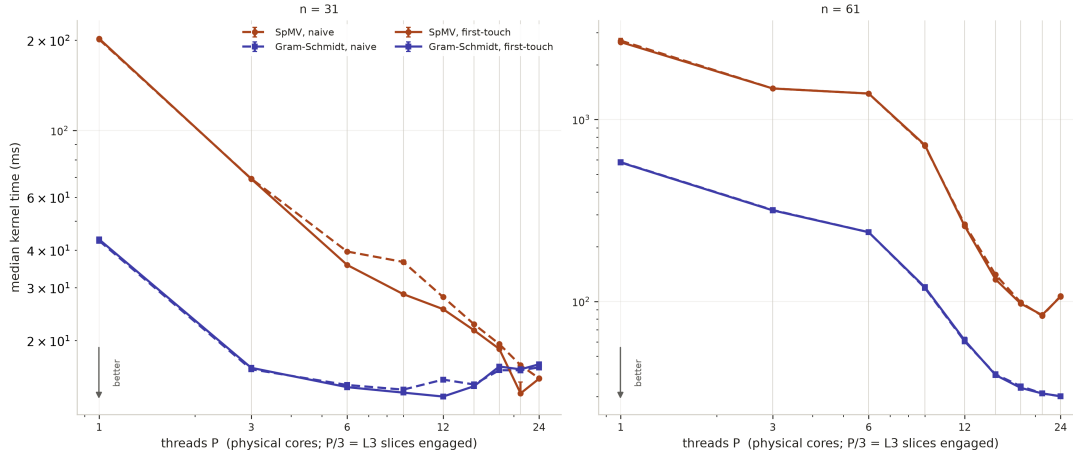


Figure 2: Per-kernel strong scaling at $n = 31$ and $n = 61$, both initialization arms. SpMV shows a plateau at $n = 31$ and a superlinear transition at $n = 61$; Gram-Schmidt shows the synchronization staircase.

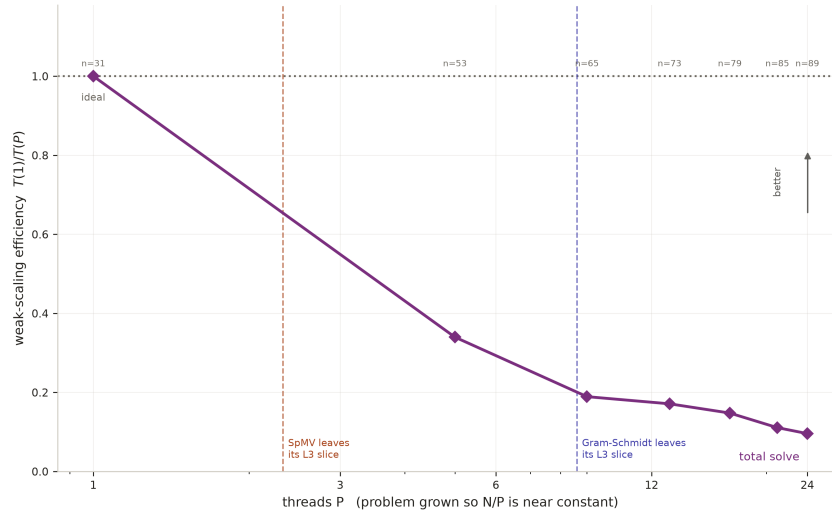


Figure 3: Per-kernel weak-scaling efficiency $T(1)/T(P)$ with $n(P) = n_1 P^{1/3}$ rounded odd, cache-tier crossings annotated.

quadratic share (Table 30) and supplies the measured range used by the small- m discussion in Section 7.4.

6.3 The locality control

In this subsection, the monospace words block, cyclic, and rotate are names of CPU row schedules, not Krylov blocks or CUDA thread blocks.

At $n = 61$ the SpMV runtime drops *superlinearly* ($36.7\times$ on 24 cores) with a knee at $P = 12$ (Figure 2). Cache residency provides the candidate explanation. Once each CCX's share of the matrix ($3 \times 52.8/P$ MiB) fits in its own 16 MiB L3 slice and stays resident across repeated SpMVs, reads come from L3 instead of DRAM. The implied aggregate bandwidth crosses the independently measured socket DRAM ceiling (~ 95 GB/s) at $P = 12$, ruling out DRAM alone

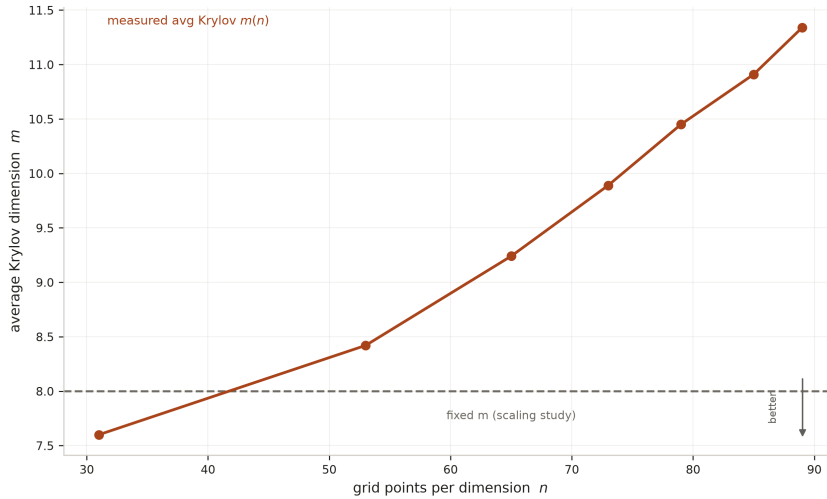


Figure 4: The $m(n)$ Krylov-growth curve, measured with the convergence check enabled and quarantined from the scaling instrument, which pins $m = 8$.

and supporting service from the cache hierarchy.

The initialization arms cannot test this explanation because puffin exposes the 3960X as one NUMA domain and Zen 2 uses a per-CCX victim L3 cache. AMD calls the firmware-controlled subdivision of a socket into software-visible NUMA domains *NUMA Per Socket* (NPS) [16]. NPS is selected in BIOS/UEFI, and firmware access was unavailable on puffin, so it could not be added as an experimental arm. In the available topology, the core that *accesses* a row determines L3 residency, not the thread that first touched its page. Both initialization arms use the same row partition and therefore produce identical curves. The controlled experiment changes only the compute schedule, and with it the rows read by each core (Table 7, Figure 5).

Table 7: Observed SpMV behavior per compute schedule in the locality control at $n = 61$ (speedup at $P = 24$; implied aggregate bandwidth at $P = 12$).

Schedule	What it does	Observed behavior
block	contiguous band per core, fixed across SpMV	$36.7\times$, knee at $P = 12$, 176 GB/s
cyclic	cache-line-sized row chunks dealt round-robin, fixed	$7.0\times$, no knee, 36 GB/s
rotate	contiguous bands, rotating by one each SpMV	$6.6\times$, no knee, 66 GB/s

The cyclic schedule was expected to keep the transition because it leaves the per-CCX byte *volume* unchanged. Instead, it collapses to the rotate curve: $7.0\times$ versus $6.6\times$, compared with $36.7\times$ for block. **Contiguity, not volume alone, keeps a slice resident.** This is the locality property that a matrix-powers tile attempts to reuse.

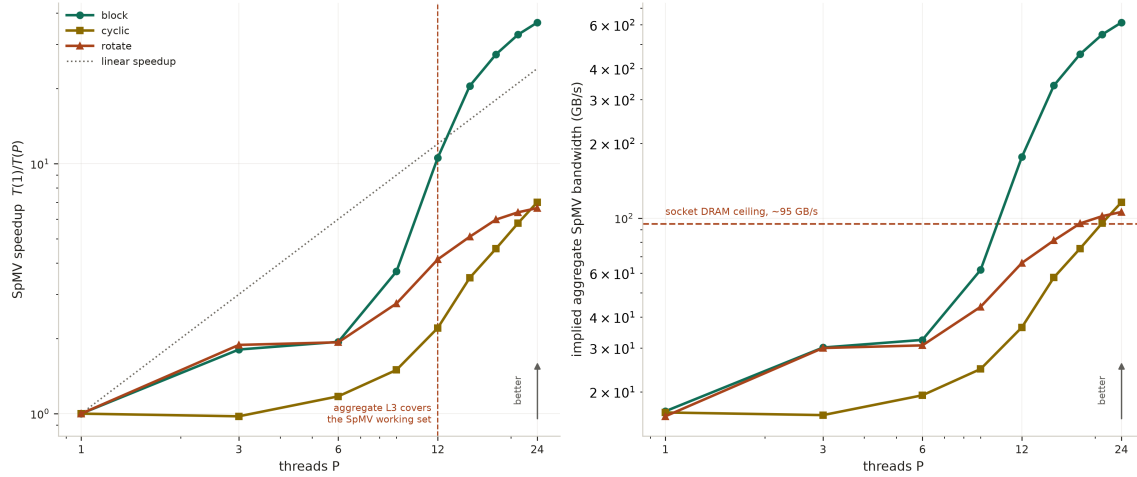


Figure 5: The locality control. Only the block row schedule carries an aggregate bandwidth DRAM cannot supply; the implied traffic above the measured DRAM roof must therefore be served from L3 or a nearer cache tier.

7 Regime analysis

The scaling study shows that the baseline solver is limited by memory traffic and synchronization. It does not show where a CA reformulation will help. The regime analysis organizes this question with a dimensionless map whose axes represent the two communication mechanisms. Each unit line marks demand comparable with supply, so it is an opportunity boundary rather than a universal performance crossover. A synthetic instrument traverses the map first. The real Black–Scholes operator is then placed on it and compared with the measured outcomes.

The synthetic instrument extends the discretized operator of Section 2 into a family of independently controlled parameters. Its backbone is a *Kronecker sum*. A tridiagonal operator $\mathbf{T} \in \mathbb{R}^{n_1 \times n_1}$ combines advection and diffusion using the same three-point stencil as Section 2, with diffusion scale σ and advection strength γ . It extends to the d -dimensional grid as

$$\mathbf{A}_d = \mathbf{T} \oplus \cdots \oplus \mathbf{T} = \sum_{i=1}^d \mathbf{I} \otimes \cdots \otimes \underbrace{\mathbf{T}}_{i\text{-th factor}} \otimes \cdots \otimes \mathbf{I} \in \mathbb{R}^{n_1^d \times n_1^d},$$

This applies the same one-dimensional stencil along each axis, as in a separable PDE discretization [17]. Two optional terms break the clean structure in controlled ways that mirror the real operator in (1). The *correlation cross term* adds $c(\mathbf{D} \otimes \mathbf{D})$, a discrete mixed second derivative coupling one axis pair and playing the role of the PDE’s $\rho_{dd'}$ terms. Turning it off gives an axis-by-axis factorization with a closed-form spectrum; turning it on breaks that factorization. The *symmetric scatter permutation* relabels the grid unknowns by replacing $\mathbf{A}$ with $\mathbf{PAP}^\top$. This destroys the banded memory layout while preserving the operator’s numerical properties, so locality can be changed without changing the mathematical problem (gate C2 below).

The parameters move the map axes independently. Grid size $N = n_1^d$ sets the Arnoldi working

set and therefore R_v . The spectral parameters (σ , μ , γ , and ρ) change basis conditioning and the width admitted by the implemented guard without changing the memory footprint. The scatter parameter changes the access pattern at a fixed spectrum. This separation lets one operator family move one modeled coordinate while holding the other fixed, making it an instrument rather than only a test matrix.

7.1 The coordinates

Two coordinates separate working-set pressure from reduction latency.

Table 8: The two dimensionless coordinates.

Axis	Definition	Mechanism it governs	Scales like
R_v	Arnoldi working set / aggregate last-level cache	vertical mechanism , matrix-powers tiling to avoid DRAM traffic	$\sim 1/P$
R_h	reduction cost / compute per core between reductions	horizontal mechanism , s -step to cut the reduction count	$\sim P \log P$

Each coordinate is a demand-to-supply ratio. R_v divides the Arnoldi working-set bytes by the last-level-cache capacity available to hold them. R_h divides global-reduction latency by the useful computation between reductions. The unit lines $\theta_v = \theta_h = 1$ mark equality of these modeled quantities. Below a line, the corresponding resource is not an evident first-order opportunity. Above it, a CA mechanism may have communication to remove. The ratios do not include associativity, reuse distance, halo work, launch costs, overlap, or topology, so they cannot locate the wall-clock crossover by themselves. The four regions are therefore labeled by *opportunity*: neither, horizontal, vertical, or both. The Krylov dimension m changes the working set and R_v , but two machines with the same m can still fall on opposite sides of the unit line.

The numerator of R_h needs a measured magnitude, not only a trend in P . A calibration instrument therefore times an isolated scalar all-reduce with no application work. The scalar itself is the payload. The fitted model is

$$t_{\text{reduce}}(P) = t_{\text{intra}} \cdot \text{intra levels}(P) + t_{\text{cross}} \cdot \min(\text{cross levels}(P), L^*). \quad (6)$$

On puffin, $t_{\text{intra}} = 120.8 \text{ ns}$, $t_{\text{cross}} = 652.8 \text{ ns}$, and $L^* = 2.20$. A cross-CCX tree level costs $5.4 \times$ an intra-CCX level, and subtree concurrency hides additional cost beyond roughly two cache-domain crossings. Measured tree all-reduce latency runs from 9.6 ns at $P = 1$ to $1.70 \mu\text{s}$ at $P = 24$ ($R^2 = 0.98$ against the model). The cost is set by how many cache-domain boundaries the tree crosses rather than core count, which is why this coordinate cannot be measured on a

virtualized topology (Section 4).

7.2 The instrument and its gates

The map depends on the algebra of the synthetic instrument, so a single-threaded control program checks its arithmetic claims before any timing begins (Table 9). The program contains no timed result and does not depend on the host. Its verdicts are therefore properties of the operator, not the machine. A *gate* checks a claim required by the downstream study; a *finding* records a result without making it a prerequisite.

Table 9: The controls. A gate failure is fatal; a finding is a result.

Control	Claim under test	Status
C1	the Kronecker-sum spectrum is analytic and correct	gate
C2	the scatter knob is an exact similarity transform (so a pure- R_v dial exists)	gate
C3	measured m obeys the Hochbruck–Lubich spectral bound [9]	gate (Hermitian PSD)
C3b	a spectral shift leaves m fixed	finding
C4	κ of the matrix-powers basis equals the row-scaled Vandermonde from the spectrum	gate when normal
C5	relation of the implemented $u^{-1/2}$ guard to observed factorization behavior	finding
C5b	worst-case Krylov-block count over the physical ensemble	finding
C6	where the operator is non-normal, does the spectral prediction survive?	finding

C2 is essential because it separates layout from spectrum. The scatter parameter applies $\mathbf{PAP}^\top$, where $\mathbf{P}$ is a permutation matrix. Since $\mathbf{P}$ is orthogonal, $\mathbf{PAP}^\top = \mathbf{PAP}^{-1}$ is a similarity transform, and similar matrices have the same spectrum [17, 18]. The permutation changes which rows are adjacent in memory without changing the spectrum. If both changed together, either could explain a timing difference; that would be a confound. The sweep then proceeds through five phases: gates, separation controls, the m range, non-normality, and the real Black–Scholes operator with its discounted payoff across grids. The final phase places the real operator on the same (R_v, R_h) plane as the synthetic instrument and tests transfer by measurement.

7.3 Basis conditioning and the confound control

The regime instrument records $s_{\max}$, the largest polynomial degree admitted by the implemented $u^{-1/2}$ conditioning guard. The corresponding Krylov block has $q_{\max} = s_{\max} + 1$ columns. A row-scaled Vandermonde construction predicts this *heuristic* degree from the spectrum [18], and the experiment compares the prediction with the guard. Where a closed-form spectrum

exists, the two integers agree on 20 of 21 sampled runs. The exception is the most non-normal point ($\gamma = 0.4$, $\kappa(X) = 7.7 \times 10^8$), where they differ by one degree. This agreement describes the sampled predictor–guard relationship; it is not a stability theorem or a population-level accuracy claim. The real Black–Scholes points differ by at most one degree over the measured grids.

The first separation test asks whether guard-admitted $s_{\max}$ simply tracks the Krylov dimension m . If it did, Krylov-block width could not be predicted or tuned independently, and a longer solve might be mistaken for poor conditioning. For a normal operator $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^*$, the matrix-powers Krylov block factors as

$$[\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^s\mathbf{v}] = \mathbf{X} \operatorname{diag}(\mathbf{X}^*\mathbf{v}) \mathbf{V}_s(\lambda_1, \dots, \lambda_N),$$

where $\mathbf{V}_s$ is the Vandermonde matrix in the eigenvalues. Conditioning depends on the eigenvalues and the starting vector’s eigen-coefficients; m does not appear [5, 18]. The measurements follow this separation. With the spectrum fixed and m varied from 5 to 12 through step size and tolerance, $s_{\max}$ remains 7. With m fixed at 5 and the spectrum scaled and shifted, $s_{\max}$ ranges from 4 to 14. Solver length and admissible Krylov-block width are therefore treated as separate inputs. Orthogonality and solution diagnostics provide the downstream numerical checks.

7.4 The reduction ledger in the measured small- m regime

Once a usable Krylov-block width is known, the next question is how much synchronization it removes from one Arnoldi cycle. The ledger compares the orthogonalizers used by the two principal arms. The baseline integrator uses MGS (Algorithm 2), while the CA arm uses CholeskyQR2. CholeskyQR2 orthogonalizes a complete Krylov block with two Gram-formation phases, independent of the block width. Its appeal here is the communication count and measured cost; the reach of its sufficient bound is assessed separately. TSQR obtains a similar synchronization count through a reduction tree, but with greater implementation complexity [2, 3]. It is measured later as a robustness comparison.

The *reduction ledger* in Table 10 counts global reductions in one Arnoldi cycle. MGS requires $1 + m(m + 3)/2$ reductions, while a Krylov-block method with $q_{\max}$ columns requires $1 + 2\lceil m/q_{\max} \rceil$. The guard-admitted value $q_{\max} = 8$ gives an **11–16× reduction-count cut** across the measured $m = 8$ –11 range. Here “small- m ” describes the observed number of Krylov blocks; it is not a numerical safety theorem. The count is exact once the width is fixed. Safety is checked separately through successful factorization, orthogonality, and referee agreement. The real operator uses $m = 6$ –8 on eigensolvable grids $n = 10$ –20 and mean $m = 7.6$ –11.3 on $n = 31$ –89, so one reported cycle contains only two or three Krylov blocks.

Table 10: Conditional reduction ledger at $q_{\max} = 8$ Krylov-block columns.

m	MGS reductions	s -step + CholQR2	cut
5	21	3	$7.0\times$
8	45	3	$15.0\times$
9	55	5	$11.0\times$
11	78	5	$15.6\times$
12	91	5	$18.2\times$

7.5 Empirical log-sensitivity to non-normality

The spectral prediction of Krylov-block conditioning assumes a normal operator. The pricing operator is non-normal because of convection and mixed derivatives. Prediction error is reported as $|\log_{10} \kappa_{\text{meas}} - \log_{10} \kappa_{\text{pred}}|$ in *decades*. One decade is a factor of 10, and 0.3 decades is approximately a factor of 2. In the sampled family, each added matrix power raises the measured condition number by a median 1.05 decades. A difference of roughly 1.2 decades is therefore comparable with one change in guard-admitted degree.

Figure 6 tests whether increasing eigenvector non-orthogonality makes the spectral predictor less accurate. Prediction error grows more slowly for constant-coefficient operators than for the variable-coefficient advection arm. The fits are descriptive and are not extrapolated. The real Black–Scholes points are excluded from both fits; they differ from prediction by 0.19–0.85 decades at measured $\kappa(X) \leq 2.3 \times 10^3$. This is less than one sampled change in guard-admitted degree. The predictor remains useful over the measured grids, while direct factorization and orthogonality checks determine acceptance.

Operator by operator, what the spectral prediction needs is normality; definiteness and separability are conveniences (Table 11).

Table 11: Which structural properties the spectral prediction of $s_{\max}$ actually needs.

Operator	Normal?	Definite?	Separable?	Spectral prediction
Laplacian	yes	PSD	yes	exact (closed form)
shift μ	yes	indefinite	yes	exact ($\kappa(X) = 1$)
correlation ρ	yes	PSD	no	exact (dense eigensolve)
constant γ advection	no	–	yes	agrees within 1 degree
$\rho + \gamma$	no	–	no	agrees within 1 degree
variable γ advection	no	–	no	differs by >1 degree

The rows apply cumulative changes to one backbone. The Laplacian is a pure-diffusion Kronecker sum with a tridiagonal Toeplitz one-dimensional factor, meaning that each of its three diagonals is constant. The shift adds $\mu\mathbf{I}$ and moves the eigenvalues without changing the eigenvectors. The correlation row adds $c(\mathbf{D} \otimes \mathbf{D})$, and the advection rows add a skew-symmetric first difference with constant or position-dependent strength γ . The three structural properties

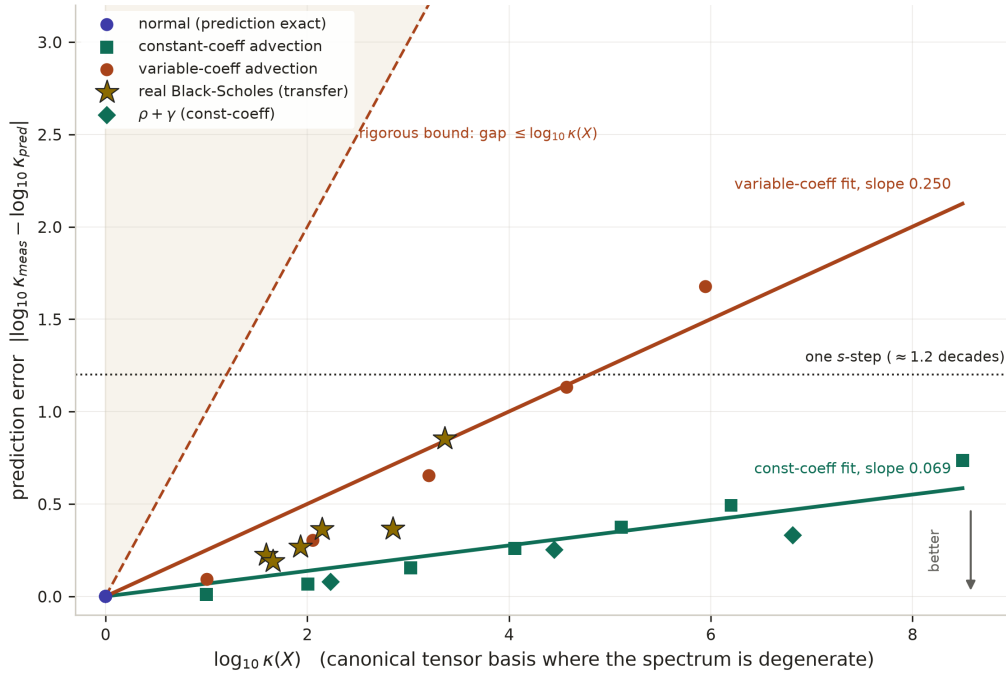


Figure 6: Empirical log-sensitivity of prediction error in decades against $\log_{10} \kappa(X)$. The identity line is the worst-case eigenvector-conditioning envelope; separate descriptive fits are shown for the sampled constant- and variable-coefficient families. The real Black–Scholes operator is a test point excluded from both fits.

are standard [17, 18]. Normality, $\mathbf{A}\mathbf{A}^\top = \mathbf{A}^\top\mathbf{A}$, supplies an orthonormal eigenbasis. Definiteness places a symmetric operator’s eigenvalues on one side of zero, and separability gives a Kronecker-sum factorization. Normality makes the Vandermonde prediction exact. Separability determines whether the spectrum comes from small factors or requires a dense eigensolve. The last column compares predicted and measured guard degrees; factorization and orthogonality provide the acceptance evidence.

7.6 Conditioning growth with asset dimension

At fixed $d = 3$, mesh refinement leaves $\kappa(X)$ approximately flat in the synthetic design because the cell Péclet number, the local convection-to-diffusion ratio, is held approximately constant [7]. Asset dimension is the more important growth axis. With one coupled axis pair, the operator factors as $\mathbf{A}_d = \mathbf{B}_{01} \oplus \mathbf{T} \oplus \cdots \oplus \mathbf{T}$. The Kronecker-product eigenvectors then make condition numbers multiply [17]:

$$\kappa(X) = \kappa(X_{01}) \kappa(X_1)^{d-2} \quad \implies \quad \log_{10} \kappa(X) = 0.40499 d + 0.01422. \quad (7)$$

The slope $0.404995 = \log_{10} \kappa(X_1)$ comes from one small one-dimensional eigensolve, not from a fit over d . It reproduces the nondegenerate $d = 2, 3$ points to 3×10^{-7} . From $d = 4$ onward, exact axis symmetries create repeated eigenvalues, and the dense eigensolver may choose dif-

ferent bases inside the same invariant subspace. Across eight symmetric permutations, the ratio between the largest and smallest dense $\kappa(X)$ is 3.84 at $d = 4$ and 7.76 at $d = 5$. The canonical Kronecker-factor value therefore provides the interpretable dimension trend (Table 12).

Table 12: Canonical vs. dense $\kappa(X)$ along the asset dimension.

d	N	canonical $\kappa(X)$	dense $\kappa(X)$	spectrum
2	16	6.671	6.671	simple (agree to 3×10^{-7})
3	64	16.95	16.95	simple (agree to 3×10^{-7})
4	256	43.07	109.5	repeated; dense basis-dependent
5	1024	109.4	5.2×10^3	repeated; dense basis-dependent
6	4096	278.1	3.7×10^4	repeated; dense basis-dependent

Within this synthetic design, changing ρ over the sampled range changes canonical conditioning by about as much as adding 1.7 assets, while dependence on d remains geometric. The *Krylov-block width* here is the number q of columns—consecutive matrix powers—orthogonalized in one communication phase. A width selected at low asset count should not be reused at higher d without new conditioning and accuracy measurements.

For a pricer, two consequences matter. First, increasing asset count can require a narrower stable Krylov block even when the one-dimensional mesh is unchanged. Second, the full grid contains n_1^d states and becomes impractical rapidly. Leentvaar and Oosterlee address this dimensional growth by combining solutions on smaller component grids to approximate the full-grid basket price, using coordinate transformations and grid stretching to retain accuracy through five assets [19]. Monte Carlo and quasi-Monte Carlo with dimension reduction provide another route at still higher dimension [20, 21]. The tensor conditioning identity is specific to the synthetic one-pair coupling; the real basket operator couples all asset pairs and still requires direct numerical checks.

7.7 The horizontal mechanism

Cutting the reduction count by 11–16 $\times$ leaves Gram–Schmidt wall-clock time unchanged. Two measurements explain why.

1. **The reduction primitive was not the bottleneck.** The synchronization staircase in Figure 2 and MGS’s repeated global dot products made the primitive an obvious candidate constraint before the control was run. Replacing the naive barrier-plus- $O(P)$ scan with a binary tree improves bare all-reduce latency by 1.7 $\times$ at $P = 24$ (2.91 μ s to 1.70 μ s) but leaves Gram–Schmidt wall-clock within 5% at every core count. The OpenMP barrier is already tree-structured, and the removed scan was not the dominant term.
2. **Gram–Schmidt is vector-bound, not reduction-bound.** MGS re-reads column V_i once for every $j \geq i$, so $m(m+1)/2$ column reads at $8N$ bytes each. That quadratic is a

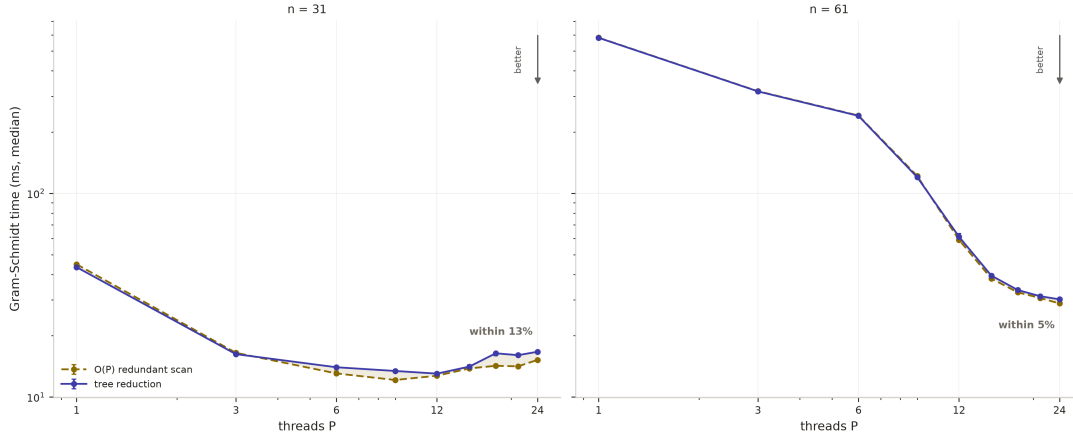


Figure 7: Gram–Schmidt wall-clock under the calibrated tree reduction vs. the $O(P)$ redundant scan it replaced ($n = 61$, arm B; one contaminated $P = 15$ launch, inflated across all kernels, is excluded). A $1.7\times$ faster primitive leaves the kernel unchanged within 5%.

bandwidth term, and it is what the roll-off in the scaling study is actually made of.

Before the timed experiment, the calibrated map placed the production-resolution case at $R_h = 0.080 \ll \theta_h = 1$. Reduction latency was only 8% of useful compute time between reductions, so the case did not enter puffin’s upper-right opportunity region. The later null timing result is consistent with this pre-run placement. The unit line still marks equality of modeled terms, not a universal wall-clock crossover, because the ratio omits vector traffic, overlap, and implementation overhead.

No measured or calibrated puffin configuration enters the upper-right opportunity region. Puffin is a single-NUMA socket, so its reduction tree crosses no boundary more expensive than a CCX. In the present model, reaching $R_h = 1$ while $R_v > 1$ would require a reduction cost roughly $25\times$ above the calibrated range. This conclusion is specific to puffin’s topology.

7.8 The vertical mechanism

The roofline in Section A.3 identifies $4\text{--}8\times$ of recoverable traffic for the spilled SpMV, but (10) is only a ceiling. It counts the bytes that tiling could save and omits the redundant halo arithmetic required to save them. A timed sweep therefore compares a cache-blocked matrix-powers kernel with the untilted baseline on grids spanning $R_v = 0.035$ to 86. Measured performance remains well below the modeled traffic ceiling (Table 13).

The tiled arm never becomes bandwidth-bound, so the gap points to a kernel ceiling rather than a measured performance boundary. A controlled no-halo diagnostic skips ghost-region work. It cannot produce a valid basis or solver result and is used only to isolate the halo cost (Figure 8). At L2, where small panels have a large halo, removing it raises throughput from 1.17 to 2.13. The untilted kernel reaches 2.70 at the same $n_1 = 400$ point. Cache blocking recovers traffic, but the redundant halo arithmetic costs more than it saves. At the corresponding L3 point,

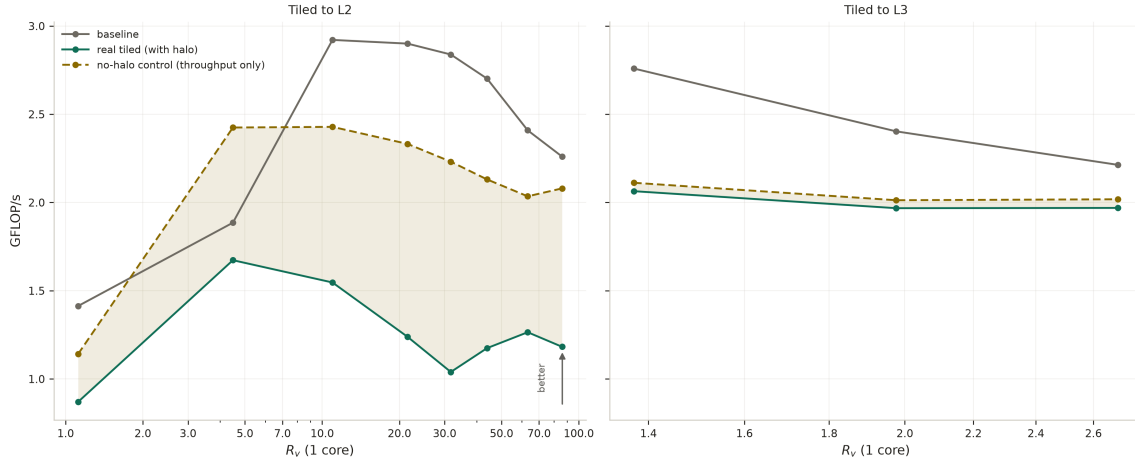


Figure 8: The no-halo diagnostic. L2 tiling recovers toward baseline without the halo (the halo trade is the shortfall); L3 tiling barely moves (the cap is insensitive to cache residency).

removing the thinner halo changes throughput only from 2.06 to 2.11, compared with 2.76 for the matched untiled baseline. The persistent L3 cap is most consistent with gather latency from indirect column access.

Table 13: Measured speedup of the tiled matrix-powers kernel against the modeled traffic ratio in the vertical sweep.

Pattern	Tile level	R_v (1 core)	Measured speedup	Modeled ratio
banded	L3	0.035–2.69	0.72–1.31 ×	4–8×
banded	L2	1.12–86.1	0.37–0.89 ×	4–8×
scattered	L3	0.035–2.69	1.00×	4×

The scattered arm tests whether the geometric tile survives a change in storage order. The permutation destroys the regular band and halo layout, so the code falls back to the naive form and measures 1.00×

 by construction. Since the similarity transform keeps the spectrum and working set fixed, the comparison isolates sensitivity to storage order. Matrix-powers block-
ing is still possible for general sparse matrices. For example, the recursive algebraic coloring engine (RACE) uses breadth-first graph levels to recover spatial and temporal locality without assuming a geometric band layout [22].

Puffin’s uncore counters require root access, so this experiment does not count DRAM traffic. The modeled bytes are instead reconciled against the roofline, which provides weaker evidence and is labeled accordingly.

7.9 Krylov-block width

The CPU regime instrument pins the Krylov block at the largest degree admitted by the implemented conditioning guard. The experiment then asks which limit arrives first as the block grows: numerical conditioning or shared-cache ghost capacity. A wider Krylov block gives

fewer reductions and more powers per operator stream, but two independent ceilings restrict it.

- **Numerical conditioning.** $\kappa([\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^{q-1}\mathbf{v}])$ grows rapidly with q . Beyond the heuristic $u^{-1/2}$ line, the current implementation narrows the Krylov block. This is a software policy, not the dimension-dependent sufficient condition in (5).
- **Ghost capacity.** The halo grows with recurrence depth, so a wider Krylov block shrinks the panel until none fits and the tiling collapses.

The first limit depends on both the operator and the machine. On puffin, it also depends on the tiling level (Table 14).

The instrument denotes by s the highest power in $[\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^s\mathbf{v}]$, so a degree- s Krylov block contains $s + 1$ columns. The sweep covers every integer $s = 1, \dots, 39$, matching the common $m_{\max}$, and records the numerical and capacity stops independently. The numerical limit depends on grid refinement rather than cache level: $n_1 = 420$ admits $s = 8$, whereas the broader discrete spectra at $n_1 = 480$ and 560 reach the conditioning guard at $s = 6$. Agreement between L2 and L3 at fixed n_1 is therefore expected; only the capacity limit changes with tile level.

Table 14: Observed Krylov-block degree limits over $s = 1, \dots, 39$. The numerical column records the operational $u^{-1/2}$ guard rather than the dimension-dependent sufficient limit.

Tile level	n_1	Numerical limit s	Capacity limit s	First observed limit
L3	420	8	23	heuristic guard
L3	480	6	21	heuristic guard
L3	560	6	19	heuristic guard
L2	420	8	1	capacity
L2	480	6	1	capacity
L2	560	6	1	capacity

The distinction matters because the two limits respond to different changes. The numerical guard depends on how quickly the Krylov-block condition number grows with each power, which in turn depends on the polynomial basis.

Newton and Chebyshev bases are standard approaches to moderating that growth [2, 5, 23]. Where the guard binds first, a better-scaled basis may admit a wider Krylov block, reducing the number of orthogonalization phases and moving the solver leftward in reduction cost per useful column. It can also change the recurrence coefficients, Hessenberg assembly, matrix-powers work, and ghost traffic, so both R_v and R_h must be recomputed rather than carrying the monomial coordinates over unchanged. Where ghost capacity binds first, a better basis alone cannot enlarge the tile. None of these performance possibilities supplies a theorem for distributed CholeskyQR2 stability.

7.10 Placing the real operator on the map

The regime analysis becomes useful for the pricer once the real operator is placed from measurements rather than a synthetic surrogate. At every production grid in Table 15, a placement-only run constructs the sparse Black–Scholes operator, measures the Krylov dimension m at $h = 0.01$ and tolerance 10^{-8} , and evaluates (R_v, R_h) with puffin’s calibrated constants. Only the dense eigensolve diagnostics are omitted. They are not needed for the coordinates and are infeasible at the larger state dimensions.

Grid refinement moves the coordinates in opposite directions. From $n = 25$ to 120, m rises from 9 to 41, and the larger basis and sparse operator drive R_v from 0.0356 to 7.377. During the same interval, useful local work grows faster than calibrated reduction latency, so R_h falls from 1.017 to 0.00464. The measured path starts in the horizontal-only region at $n = 25$, passes through the region where neither mechanism is indicated for $n = 30$ –61, and enters the vertical-only region at $n = 74$. For this CPU and operator, the map points the production experiment toward matrix-powers reuse. The timed vertical sweep then tests whether the implementation recovers that opportunity.

The Krylov growth also has the expected scale. A descriptive fit over these production measurements gives $m \propto n^{0.95}$. Since $\|\mathbf{A}\|$ grows approximately as n^2 , the trend $m \sim \sqrt{h\|\mathbf{A}\|}$ is consistent with the diagnostic scale for polynomial exponential approximation [9]. The operator is non-Hermitian and non-normal, so this is an explanation of the observed trend rather than a convergence theorem, and no extrapolation beyond $n = 120$ is claimed.

Table 15: Measured placement of the real Black–Scholes operator on puffin at $h = 0.01$ and tolerance 10^{-8} . The unit lines $R_v = 1$ and $R_h = 1$ define the named opportunity regions.

n	N	m	R_v	R_h	Opportunity
25	15,625	9	0.0356	1.017	horizontal only
30	27,000	10	0.0636	0.623	neither
40	64,000	12	0.160	0.292	neither
50	125,000	15	0.336	0.171	neither
61	226,981	18	0.653	0.1057	neither
74	405,224	22	1.266	0.0671	vertical only
90	729,000	27	2.499	0.00863	vertical only
120	1,728,000	41	7.377	0.00464	vertical only

The transition at $n = 74$ marks the first sampled grid where working-set pressure makes matrix-powers reuse worth testing. It does not promise a speedup. The timed sweep shows that halo arithmetic and access geometry can prevent the kernel from realizing the traffic opportunity. Puffin has no upper-right region in the calibrated range because reaching it would require a reduction roughly $25\times$ more expensive.

Collectives can reach costs of this order when they cross NUMA, socket, device, or node

boundaries. This is why s -step and pipelined methods are often studied on distributed systems [1, 2, 3, 5, 24, 25]. Synge has two nodes with two Tesla V100s per node, PCIe within each node, and a network fabric between nodes. It therefore provides several distinct communication boundaries. The next section recomputes the opportunity coordinates for this memory system and compares them with measurements, without assuming that the CPU unit lines transfer as GPU performance crossovers.

8 Recomputing the map for a GPU

8.1 A cross-architecture transfer test

Because R_v and R_h are dimensionless, the same definitions can be evaluated for another pairing of an operator and a machine. Moving from puffin to a GPU changes both cache ownership and reduction mechanisms. This section recomputes the machine-dependent terms and asks two questions: do the opportunity coordinates remain interpretable, and does measured kernel behavior agree with their placement? A disagreement can reveal a missing term in the model.

The two GPU classes provide a useful architectural contrast (Table 16). The RTX 3090 and V100 have the same nominal 6 MB L2 capacity and similar measured streaming bandwidth, while their FP64 peaks differ by about $11\times$. They also differ in SM organization, cache behavior, clocks, memory technology, ECC, scheduling, and launch behavior. The comparison can expose a roofline distinction for the measured kernels, but it cannot isolate FP64 throughput as the cause.

Table 16: Two GPU architectures measured on idle devices. Memory bandwidth is achieved (STREAM); the FP64 ridge is peak FP64 over achieved bandwidth. Similar nominal L2 capacity and measured bandwidth make the contrast useful, but architectural differences beyond FP64 remain uncontrolled.

	L2	Memory BW	FP64 peak	FP64 ridge
RTX 3090 (consumer, puffin)	6 MB	821 GB/s	0.570 TFLOP/s	0.69 FLOP/B
Tesla V100 (datacenter, synge)	6 MB	818 GB/s	6.375 TFLOP/s	7.79 FLOP/B

This contrast makes one precondition explicit: the map describes communication opportunities only when the relevant kernel lies on the memory side of its roofline. A compute-bound kernel is outside that interpretation. Each placement therefore begins with the precision-specific check $AI \cdot BW_{\text{achieved}} < \text{FLOP/s}_{\text{peak}}$. For the measured kernels, it separates the memory-bound V100 Gram kernel from the compute-bound RTX 3090 Gram kernel. This scopes the mechanism without assigning the difference solely to FP64 throughput.

8.2 What transfers, and what breaks

The synthetic operator, the tensor identity in (7), and the conditional reduction ledger are machine-independent. The CholeskyQR2 guard remains an operational policy whose sufficient theory is assessed separately. The unit lines $\theta_v = \theta_h = 1$ also remain equality lines, not portable performance thresholds. The machine-dependent terms must be recomputed, and three changes are structural rather than simple constant substitutions:

- **The denominator of R_v .** On the CPU each engaged CCX brings its own L3 slice, so aggregate last-level cache scales with the team. A GPU's L2 is one fixed cache shared by every streaming multiprocessor, and engaging more of the device adds no cache. Within one device, R_v therefore stops depending on team size and the two axes decouple. On the CPU both coordinates shared the parallelism parameter ($R_v \sim 1/P$, $R_h \sim P \log P$), which made $R_v R_h$ approximately P -independent (~ 0.04 on puffin) in the calibrated model and excluded the upper-right region. A GPU provides no structural reason for that cancellation, so the product must be re-derived from the chosen definition of team size.
- **The numerator of R_h .** The CPU model has two reduction levels and one saturation constant (6). A GPU reduction crosses at least five qualitatively different levels (Table 17), and a grid-wide reduction includes fixed kernel-launch latency with no CPU analogue. Two parameters cannot represent all five levels. The GPU model therefore needs a vector of topology costs and a predictor that traverses it. On-device, intra-node PCIe, and inter-node reductions place the same problem at different R_h values. The performance crossover is still measured separately.
- **The reuse model.** On the CPU, the scatter parameter costs a factor of a few in cache-line reuse. A GPU uses *warp coalescing* to combine adjacent requests from 32 threads into a small number of memory transactions. The same permutation can defeat coalescing and turn one access into as many as 32 transactions. The scatter axis becomes more sensitive, but its reuse model must be recalibrated.

8.3 The decoupled invariant

In the CPU model, $R_v R_h$ is approximately independent of core count P because $R_v \sim 1/P$ and $R_h \sim P \log P$ move in opposite directions. On a GPU, let P denote the engaged streaming multiprocessors. The denominator of R_v is one shared L2, while R_h uses a device-wide bandwidth roof after saturation. The GPU model therefore decouples both coordinates from P : grid size moves R_v , and reduction level moves R_h . In the memory-bound regime established by the roofline check, modeled cycle time becomes bytes divided by bandwidth, and the product

Table 17: The reduction levels for a multi-node GPU cluster with a PCIe intra-node interconnect. Every tier cost is a machine constant to be calibrated, never assumed.

Tier	Mechanism	Scope	Cost class
Warp	register shuffle	on-die	near-free, no memory traffic
CUDA thread block	shared memory + barrier	on-die	cheap, on-chip
Grid	atomics or a second launch	on-die	expensive; carries launch latency
Device	PCIe + cross-socket link	intra-node	expensive; a genuine cross-socket hop
Node	network fabric	inter-node	dominant

is

$$R_v \cdot R_h = g(m) R(m) \Lambda, \quad \Lambda = \frac{\tau \text{ BW}}{C}, \quad (8)$$

in which $g(m)$ (a byte ratio) and $R(m)$ (the MGS reduction count) are the machine-independent method terms. The dimensionless machine term Λ measures how many cache-fulls of data the memory system delivers during one reduction. The upper-right window (both $R_v \geq 1$ and $R_h \geq 1$ at a footprint under the 16 GB device memory) is non-empty exactly when this product exceeds 1, and its width in N equals the product. The 16 GB ceiling never binds because it lies two orders of magnitude beyond every reachable reduction level, so R_h closes the window from above. Setting the product to 1 gives a reduction-latency threshold of $\tau^* \approx 0.9 \mu\text{s}$ for placing the production operator in the upper-right opportunity region.

Modeled equality occurs at $\tau^* \approx 0.9 \mu\text{s}$. Isolated V100 collectives at the stated payloads calibrate the five reduction levels in Table 18, with no application work included. The same operator therefore receives five R_h values. The nominal upper-right region begins between the CUDA-thread-block and grid levels, inside one device. This is a model placement, not a measured solver crossover at τ^* . On this topology, node-to-node cost is only $1.9\times$ device-to-device cost, while the largest latency jump comes from the grid reduction’s second kernel launch. Participant count, topology, and launch costs cannot be represented by one universal node-level constant.

8.4 Reaching the corner, and the gate that discriminates

At production resolution ($n_1 = 61$, $m = 12$, polynomial degree 8), the recomputed coordinates are $R_v = 7.2$ and $R_h = 1.7$ for the selected reduction levels on both cards. The operator lies inside the upper-right *opportunity* region. The dimensionless definitions can therefore be evaluated on the GPU hierarchy after recalibration. This placement does not imply that the speedup boundary is unchanged or that both mechanisms will pay.

Table 18: The calibrated V100 reduction levels. Each level is an isolated collective at its stated scope and τ is cumulative. The modeled upper-right opportunity region ($R_v R_h > 1$) begins between the CUDA-thread-block and grid levels; this is not by itself a performance crossover.

Rung	Scope	τ (cumulative)	Corner
warp (register shuffle)	on-die	$0.42 \mu s$	closed
CUDA thread block (shared + barrier)	on-die	$0.79 \mu s$	closed
grid (second kernel launch)	on-die	$5.54 \mu s$	open
device (PCIe + cross-socket)	intra-node	$11.46 \mu s$	open
node (inter-node fabric)	inter-node	$22.09 \mu s$	open

The roofline check is essential because the coordinates assume a communication-bound kernel (Table 19). SpMV, with arithmetic intensity 0.135 FLOP/B, and MGS, at 0.375, lie on the memory side of both measured roofs. The Krylov-block method introduces a tall-skinny Gram matrix whose intensity grows with width. At the tested production width, it is memory-bound on the V100 and compute-bound on the RTX 3090. The implemented $u^{-1/2}$ guard admits this width. Since the dimension-dependent sufficient condition does not cover it, the diagnostics in Section 3.2 determine numerical acceptance. The roofline result explains why the communication interpretation applies to the V100 Gram kernel but not the RTX 3090 kernel. The coordinates alone still do not predict end-to-end speedup.

Table 19: The roofline gate at production resolution, per kernel. The baseline kernels are on-map on both cards; the s -step Krylov-block Gram is memory-bound only on the datacenter card.

Kernel (AI, FLOP/B)	V100 (ridge 7.79)	3090 (ridge 0.69)
SpMV (0.135)	on map	on map
MGS (0.375)	on map	on map
s -step Gram (~ 1.1)	memory-bound ($3.9\times$)	compute-bound

8.5 Roofline-bound kernels and the s -step crossover

Measuring this roofline distinction required kernels designed for very tall matrices with few columns. In the benchmark harness, cuBLAS dsyrk and dtrsm had low utilization for these shapes. The specialized implementations use a split-K Gram, which parallelizes the long dimension and reduces partial results [26], and a per-row triangular solve, which exploits independent row solves. Both agree numerically with cuBLAS. At $s = 8$, split-K is $2091\times$ faster than cuBLAS on the V100 and $1915\times$ on the RTX 3090. The per-row solve is $4.26\times$ and $27.39\times$ faster, respectively (Table 20). These ratios are specific to the tested shapes and harness, not general comparisons with current cuBLAS, cuBLASLt, or CUTLASS.

With roofline-bound kernels the two-card contrast becomes a hardware measurement rather than a prediction. Profiled with Nsight Compute at $s = 8$, $n = 2 \times 10^6$, the *same* Gram kernel runs

Table 20: The two hand-written tall-skinny kernels at $s = 8$. Speedups are reported separately for each card because the cuBLAS baselines differ.

Kernel	V100 speedup	V100 roof	3090 speedup and roof
split-K Gram	$2091\times$	88% DRAM (memory-bound)	$1915\times$; 79% FP64 (compute-bound)
per-row trsm	$4.26\times$	95% DRAM (memory-bound)	$27.39\times$; 94% roof (at the knee)

at 86% of DRAM throughput and 16% of compute on the V100, but at 38% of DRAM and 94% of compute on the RTX 3090.

The RTX 3090 gives the negative transfer result: once the CA method’s own Gram kernel becomes compute-bound, the communication-opportunity model no longer applies. The roofline gate catches this case by comparing arithmetic intensity with each device’s FP64 ridge before R_v or R_h is interpreted. The triangular solve lies near the RTX 3090’s compute/memory knee and is less decisive.

The final experiment times the reduction saving itself. At matched $m = 12$, the Krylov-block method replaces MGS’s 91 reductions with 5 while maintaining observed orthogonality $\|\mathbf{I} - \mathbf{Q}^T \mathbf{Q}\| \sim 10^{-15}$. On one device, the specialized Krylov-block kernels beat MGS at every tested grid on both cards. Across two GPUs, each dot product or Krylov-block Gram becomes an NCCL all-reduce. Its measured $\sim 11.5 \mu\text{s}$ latency agrees with the independently calibrated reduction level. At this topology, the five-reduction arm is faster across the tested range. This is an orthogonalizer experiment, not an end-to-end price comparison. Its behavior agrees with the calibrated horizontal opportunity coordinate, while the crossover remains a measured, topology-specific result.

9 The distributed CA exponential integrator

Section 8 recomputed the opportunity coordinates for a GPU memory system and measured both mechanisms on isolated kernels. The next step is to test a complete solve. This section evaluates a communication-avoiding exponential integrator for the pricing problem in Section 2, distributed over spatial subdomains on up to four V100s across two nodes. The main question is whether the kernel-level savings survive convergence control, the implemented conditioning guard, polynomial boundary forcing, and comparison with an independent semi-discrete referee.

9.1 The solver

The grid is divided into *slabs* along the slowest axis x_3 , with one slab per GPU. Each device owns a contiguous range of grid planes and the Krylov basis for those rows. This cut makes every off-slab dependency a plane from at most two neighboring devices. Dang, Christara and

Jackson divide their single-GPU ADI work into three-dimensional CUDA execution tiles [14]. Their tiles organize work inside one device, while the slabs here define ownership between devices. This implementation still uses three-dimensional shared-memory tiles inside each slab. A full three-dimensional multi-GPU decomposition could improve surface-to-volume ratio at larger participant counts, but it would add up to six face neighbors, edge and corner dependencies, and more complicated deep halos. With at most four devices and $O(N_1 N_2)$ slow-axis bandwidth, contiguous slabs keep the communication contract manageable.

The required off-slab planes form the *halo*, or ghost region. A device imports this data before applying the operator to its boundary rows. This is where the vertical mechanism reappears at network scale. One stencil application reaches one plane beyond the slab. A matrix-powers Krylov block of width s applies the operator $s - 1$ times and reaches $s - 1$ planes out. One *deep* exchange of those planes replaces $s - 1$ shallow exchanges. The number of bytes remains the same, as Section 9.2 confirms, but the message count falls from $s - 1$ to one. That reduction matters on a latency-dominated fabric.

The remaining solver follows Section 3.2. It applies the operator *matrix-free*, computing the 19-point stencil directly instead of storing and multiplying an assembled sparse matrix. This removes matrix-index traffic. A CUDA thread block stages an interior tile and its ghost layers in shared memory, then runs the recurrence. CholeskyQR2 orthogonalizes each Krylov block. The device evaluates the operational guard $\kappa_2(\mathbf{Z}) \leq u^{-1/2}$ and narrows any block that exceeds it. As discussed in Section 3.2, this policy targets practical normal-equations breakdown; the dimension-dependent sufficient condition and distributed Gram rounding are assessed separately [11, 27]. Local Krylov-block Gram matrices and projections are combined with NCCL all-reduces, while halos use NCCL send/receive pairs [28]. Since $m \leq 39$, the projected $\mathbf{H}_m$ is at most 39×39 , and every device can replicate the small exponential.

Correctness is checked with an instrument that shares no stencil code with the production path. The referee assembles the $N + 3$ Basket operator $\tilde{\mathbf{A}}$, or the unaugmented Rainbow operator, as a sparse matrix. It applies the matrix with the vendor library and evaluates the exponential action with the Al-Mohy–Higham Taylor method [29]. Reported arms pass the convergence and conditioning guards, preserve the forcing tail or Rainbow boundary state, and agree with an arm-matched single-GPU result and the independent semi-discrete reference within the stated tolerances. These checks provide direct evidence for the tested numerical path. Theorem-level CholeskyQR2 coverage and the total financial-error budget remain separate questions.

Device contention is checked before timing because a competing process can consume memory bandwidth, cache capacity, and streaming multiprocessors. Slurm reserves the requested Sygne nodes, and NVML provides an additional idle-device gate. Adaptive decisions are compared across ranks before each collective, so disagreement aborts instead of deadlocking.

9.2 Recurrence depth, and what a fair control costs

A Krylov block of width s contains $[\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^{s-1}\mathbf{v}]$. It requires exactly $s - 1$ recurrence applications and a deep halo $s - 1$ planes wide. Every reported performance result follows this exact-depth rule. The rule makes $s = 1$ a fair non-CA control: one ordinary Arnoldi operator application and one halo plane per basis vector, with no unused power.

Table 21: Halo planes moved per Krylov basis vector under the exact-depth rule, $n = 97$ on four V100s. The measured column comes from the solver’s exchange-depth histogram.

Rule	s	Exchanges/time step	Planes per vector	Predicted
exact-depth	1	11.45–11.48	1.000	1
exact-depth	4	6.40–6.46	1.000	1

Both widths move one halo plane per basis vector, so s -step reduces message count rather than message volume. The $s = 4$ Krylov block contains four planes of powers, from degree 0 through degree 3, and therefore needs three operator applications; each application extends the stencil radius by one plane, giving a three-plane deep halo. Across the two payoffs its 5,645–5,698 KiB per time step exceeds the $s = 1$ arm’s 5,050–5,063 KiB because a fixed depth-one transition exchange is amortized over four vectors.

9.3 Strong scaling

Strong scaling asks whether additional devices reduce latency at fixed global work. The control uses $n = 61$ (226,981 unknowns), 100 time steps, tolerance 10^{-8} , $m_{\max} = 39$, a monomial basis, and CholeskyQR2. Every value is a seven-repeat median on idle devices that pass the NVML gate (Table 22, Figure 9).

Table 22: Exact-depth strong scaling at $n = 61$, showing cycle time in ms/time step, seven-repeat medians, and CA speedup $t(s=1)/t(s=4)$.

Topology	Basket			Rainbow		
	$s=1$	$s=4$	CA	$s=1$	$s=4$	CA
One V100	2.170	2.214	0.98 ×	2.187	2.172	1.01 ×
Two V100s, one node	3.006	2.221	1.35×	2.985	2.197	1.36×
Two V100s, two nodes	3.864	2.845	1.36×	3.864	2.798	1.38×
Four V100s, two nodes	4.434	2.484	1.78 ×	4.389	2.433	1.80 ×

The exact-depth arms are at parity on one GPU. Relative to $s = 1$, $s = 4$ is 0.98× as fast for Basket and 1.01× for Rainbow. One GPU has no inter-device collectives or halo exchanges to avoid, so the wider Krylov block gives no material end-to-end advantage in this local-work control. At four GPUs, participant count also changes occupancy, launch behavior, and subdomain size, so communication cannot be assigned as the only cause of every difference.

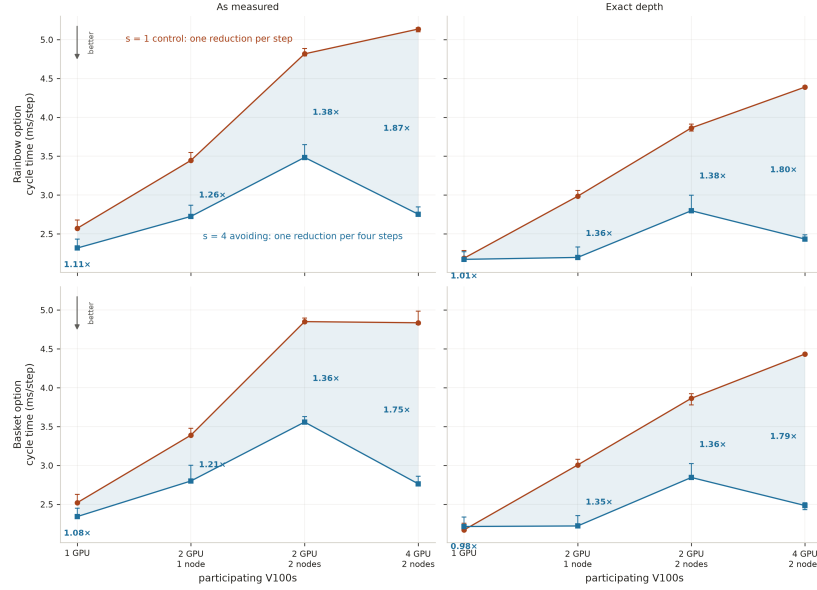


Figure 9: Strong scaling at $n = 61$ on Sygne V100s for both payoffs and both recurrence-depth controls. Marks are seven-repeat medians with the observed range. For the final exact-depth arm, the single-GPU CA ratio is near one and the four-GPU ratio is 1.78–1.80.

The distributed advantage depends on topology. Across the measured configurations, the CA ratios are 0.98, 1.35, 1.36, and 1.78 $\times$ for Basket, and 1.01, 1.36, 1.38, and 1.80 $\times$ for Rainbow. The four rungs increase monotonically, but they do not establish a participant-count scaling law or justify extrapolation beyond the tested systems. Under exact depth, the $s = 4$ shared tile holds 1,400 doubles per 128 interior points. Section 9.5 measures sensitivity to this geometry directly.

Absolute scaling at this size is poor. Four V100s at $s = 4$ deliver only 0.89 $\times$ the one-V100 performance at the same width. Strong-scaling efficiency is speedup divided by participant count, giving $0.89/4 \approx 0.22$, or about 22%. The best four-GPU time is 12–15% slower than the best one-GPU time across the two payoffs. At $n = 61$, each of the four slabs owns roughly 57,000 rows, which is too little work to saturate a V100.

The two ratios answer different questions. The algorithm-relative result is 1.78–1.80 $\times$ and asks whether $s = 4$ beats $s = 1$ on the same four-GPU topology. The 0.89 $\times$ same-width result asks whether four GPUs running $s = 4$ beat one GPU running $s = 4$. Communication avoidance helps at the measured distributed point, but distributing this small complete solve is still counterproductive.

9.4 The communication account

To interpret the difference without fitting the solver, the cost model uses independent microbenchmarks containing only communication. Bare all-reduces and halo exchanges are timed at the integrator’s payloads on idle devices for each participant count (Table 23).

Table 23: Participant calibration on idle Synge V100s across two nodes, showing the seven-repeat median latency of a bare NCCL all-reduce at the payloads the solver uses. Doubling the participant count is a real cost that a single node-tier constant cannot express.

Operation	Payload	2 participants	4 participants	Ratio
Norm	8 B	20.06 μ s	29.20 μ s	1.46 $\times$
Gram, $s=1$	8 B	20.94 μ s	28.26 μ s	1.35 $\times$
Projection, $s=1$	200 B	22.32 μ s	31.35 μ s	1.40 $\times$
Gram, $s=4$	128 B	21.91 μ s	31.16 μ s	1.42 $\times$
Projection, $s=4$	800 B	23.32 μ s	38.31 μ s	1.64 $\times$
64 MiB all-reduce	64 MiB	9.51 ms	12.51 ms	1.32 $\times$

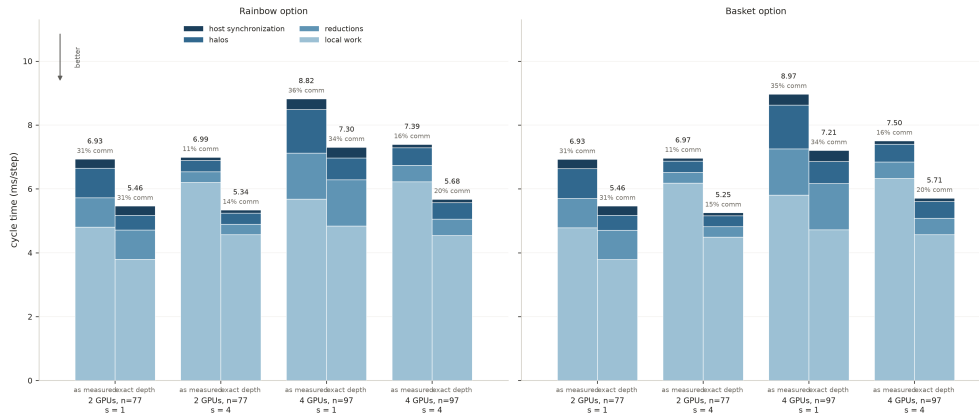


Figure 10: Cycle-time decomposition on Synge V100s across two nodes. Reductions and halos are transcript counts priced by the participant calibration; local work is the remainder, so no term is fitted. Bars carry the total cycle time and the share of it spent communicating.

Adding participants increases latency because an all-reduce must combine more contributions and traverse more links or tree/ring stages. Completion also waits for the slowest participant. On Synge, moving from two to four participants can cross both PCIe/socket and inter-node boundaries. The 8-byte rows show that cost grows even when the payload does not.

The solver transcript counts every collective and halo exchange. Each operation is priced with its calibrated latency, using the depth actually exchanged by that arm, and subtracted from the measured cycle. The remainder is local work, so no solver-time coefficient is fitted (Figure 10). In the final correction sweep, this account assigns 31–36% of the $s = 1$ cycle and 11–20% of the $s = 4$ cycle to communication. Participant count and payload explain much of the algorithm-relative difference. The non-additivity check below prevents the decomposition from being treated as a causal proof.

Agreement is close but not exact because communication costs are not additive. Summing reductions, deep exchanges, and host round-trips over-counts work that overlaps on the device. For example, the conditioning diagnostic returns to the host before deciding whether to narrow the Krylov block. At $n = 61$ on four GPUs, the corrected $s = 1$ arm performs 35.1 round-trips

per time step and stalls for 2.02 ms; the $s = 4$ arm performs 10.7 and stalls for 1.66 ms. Each host wait also drains queued device work. The measured stall is therefore an upper bound on recoverable overhead and is not added to the communication account.

Non-additivity suggests that halo transfers could be hidden behind independent interior work, a standard CUDA stream optimization. An explicit experiment compares serialized exchange with copy–compute overlap on two V100s in one node at $n = 61$, $s = 3$. The overlapped states pass the 10^{-12} agreement gate. An Nsight Systems trace shows that 99.7% of peer-halo copy time overlaps matrix powers, compared with 0% in the serialized arm. Even so, wall-clock time does not improve. The seven-repeat median changes from 2.323 to 2.379 ms per time step for Basket and from 2.304 to 2.329 ms for Rainbow, with overlapping ranges. Splitting one slab launch into an interior launch and two boundary launches triples the profiled launch count. That overhead offsets the hidden copy time, so serialized exchange remains the default.

9.5 The vertical coordinate, measured

The CPU study could not count DRAM traffic, and its tiled kernel never became bandwidth-bound (Section 7.8). On the GPU, privilege-free counters compare measured DRAM traffic with modeled demand at 44 idle-device points covering four widths, four grids, both payoffs, and four tile geometries (Table 24, Figure 11).

A spatial tile is the three-dimensional interior region assigned to one CUDA thread block: $8 \times 4 \times 4$, for example, covers 8 points in x_1 , 4 in x_2 , and 4 planes in x_3 before adding s -dependent shared-memory ghosts. Larger tiles reduce redundant ghost reads but may lower occupancy or access efficiency.

Table 24: The matrix-powers kernel against both roofs at $n = 97$ (Rainbow, full-volume $8 \times 4 \times 4$ baseline, idle V100, counter-measured DRAM traffic). Redundancy is the ratio of shared-tile reads to distinct interior points. No width comes within reach of either roof.

s	Ghost redundancy	DRAM roof	L2 roof	Verdict
1	$2.81 \times$	54.9%	3.8%	latency-bound
2	$6.00 \times$	21.1%	2.3%	latency-bound
3	$10.94 \times$	8.0%	1.6%	latency-bound
4	$18.00 \times$	2.1%	1.2%	latency-bound

The classifier separates traffic location from the kernel limiter. An earlier rule labeled a width *L2-absorbed* when measured DRAM traffic fell well below modeled tiled demand. However, a kernel reaching only 1.2% of the L2 roof is not L2-bound. Traffic attribution is therefore reported separately through the measured-to-modeled byte ratio: 3.3, 2.0, 1.1, and 0.40 as s increases from 1 to 4. The model under-counts narrow tiles because one tile row straddles several memory sectors. It over-counts wide tiles because overlap between neighboring spatial tiles remains in L2.

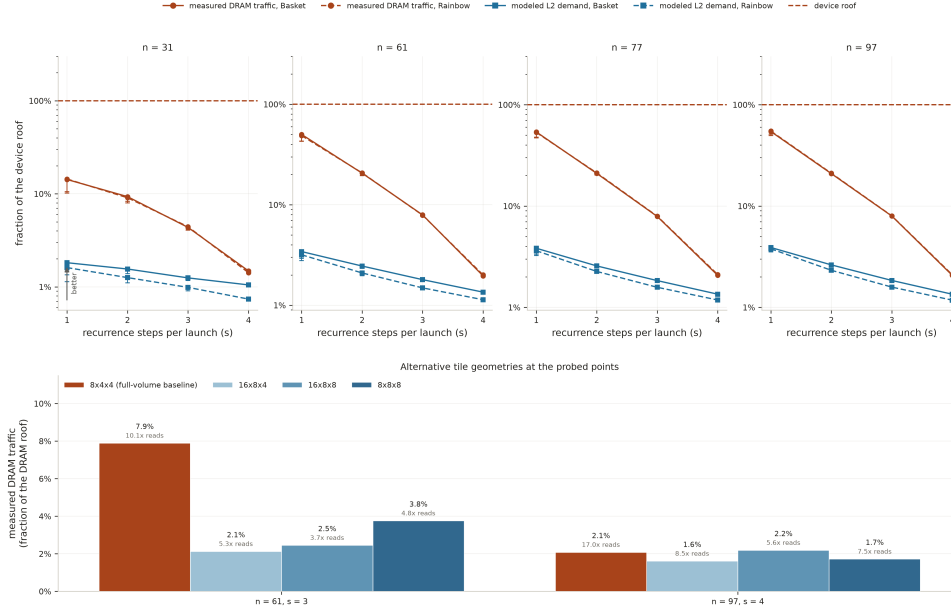


Figure 11: Matrix-powers traffic relative to the V100 device roofs (seven-repeat medians). Top: measured DRAM traffic and modeled L2 demand across widths and grids. Bottom: measured DRAM traffic for four geometries at the two tuning points; annotations give effective read redundancy.

Roof-based attribution classifies all 44 points as *latency-bound*. The largest DRAM fraction is 0.55 against a 0.75 threshold, and the largest L2 fraction is 0.04. Nsight defines *Wait* as a warp stalled on a fixed-latency execution dependency and *Long Scoreboard* as a warp waiting for an L1TEX result from local, global, surface, or texture memory [30]. Wait dominates 15 of 16 scheduler captures. Long Scoreboard leads only for Rainbow on the full-volume baseline at $n = 61, s = 3$. Occupancy is not treated as the limiter because lower-occupancy configurations sometimes run faster in this controlled sweep.

The $8 \times 8 \times 8$ and $16 \times 8 \times 8$ configurations test progressively larger volumes, while $16 \times 8 \times 4$ keeps the same 512-point interior volume as $8 \times 8 \times 8$ with a different aspect ratio. Table 25 separates volume from shape at the two canonical points.

Table 25: Tile geometry sweep on an idle V100, seven-repeat medians. Redundancy is the effective in-domain read redundancy; speedup is against the full-volume $8 \times 4 \times 4$ baseline at the same point, given as the range over the two payoffs. Occupancy is quoted for $(n=61, s=3)$ and $(n=97, s=4)$ respectively.

Tile	Occupancy	$n = 61, s = 3$		$n = 97, s = 4$	
		Redund.	Speedup	Redund.	Speedup
$8 \times 4 \times 4$ (baseline)	0.250 / 0.125	10.07×	1.00×	17.03×	1.00×
$8 \times 8 \times 8$	0.125 / 0.0625	4.81×	1.33–1.39×	7.52×	0.96–1.02×
$16 \times 8 \times 8$	0.0625	3.69×	1.10–1.21×	5.62×	1.28–1.36×
$16 \times 8 \times 4$	0.0625	5.34×	0.95–1.04×	8.45×	0.98–1.04×

Neither candidate limiter explains the fixed-geometry sweep alone. The winning geometry has *lower* occupancy than the baseline in every case; quartering occupancy makes the kernel faster.

Ghost redundancy is also insufficient by itself. At $n = 61$, the lowest-redundancy tile ($16 \times 8 \times 8$) moves the least traffic but still loses to $8 \times 8 \times 8$. At $n = 97$, the lowest-traffic tile is 30% slower than the winner. A wider tile removes redundant bytes but can reduce access efficiency, and the balance changes with the grid. No single full-volume shape won across all tested points. Selecting the best geometry separately for each grid gives only $1.08\times$ solver speedup at $s = 1$ and $1.13\times$ at $s = 4$, despite a $1.33\text{--}1.39\times$ gain for the isolated kernel, and the winning shape changes with (n, s) .

The follow-on design changes the schedule instead of choosing another fixed three-dimensional tile. A CUDA thread block owns a 16×8 tile in the $x\text{--}y$ plane and walks a contiguous z segment. It keeps a circular queue of planes for each recurrence level. Because the shared-memory footprint is independent of segment height, a longer segment amortizes the z prologue and epilogue without enlarging the resident queue. The predeclared automatic rule keeps the full-volume baseline below $n = 61$. At larger grids, it selects the segment height closest to eight z -directed CUDA thread blocks, subject to the device's opt-in shared-memory limit. An inadmissible recurrence width falls back instead of failing.

Table 26: Promoted plane-streamed cases on an idle V100. Speedup is full-volume time divided by streamed time at fixed financial contract, grid, basis, and recurrence width. Kernel and solver values come from unprofiled runs; registers, dynamic shared memory, and executed-instruction reduction come from the shortlisted Nsight captures. Resource columns are full-volume to streamed.

Contract	Point	L_z	Kernel	Solver	Registers	Shared (KiB)	Inst. cut
Basket	$n = 61, s = 3$	8	$1.591\times$	$1.204\times$	90 to 92	21.9 to 38.5	$2.90\times$
Rainbow	$n = 61, s = 3$	8	$1.657\times$	$1.222\times$	90 to 92	21.9 to 38.5	$2.88\times$
Basket	$n = 97, s = 4$	16	$1.610\times$	$1.273\times$	90 to 95	36.0 to 60.0	$3.45\times$
Rainbow	$n = 97, s = 4$	16	$1.656\times$	$1.297\times$	90 to 95	36.0 to 60.0	$3.43\times$

Held-out cases test whether the selected rule overfits its tuning points. On the selection cases, kernel and solver geometric means are $1.628\times$ (worst $1.591\times$) and $1.248\times$ (worst $1.204\times$). The four active held-out cases, both $n = 77$ monomial payoffs and Basket Chebyshev and Newton at $n = 61$, give a $1.239\times$ solver mean and $1.182\times$ worst case. Four small-grid fallback controls give $0.991\times$ and $0.974\times$. Their timing ranges all overlap the full-volume controls because both requests resolve to the same family. Every execution passes validation. The active cases support transfer, while fallback overhead remains below run-to-run resolution.

The counters support the intended mechanism. Streaming reduces the launch from 2,048 to 256 CUDA thread blocks at $n = 61$ and from 8,125 to 637 at $n = 97$. Executed instructions fall by $2.88\text{--}3.45\times$, and global-load sectors fall by $2.57\text{--}4.22\times$. This happens even though achieved occupancy roughly halves and each CUDA thread block uses more registers and shared memory. Long-Scoreboard stall samples fall in every profiled case. Bank conflicts do not move consistently and are not claimed as the cause. A separate compilation-flag screen finds no competing compiler explanation. Vectorization and fast-math produce unchanged normalized

device code. Cache-policy and register-cap variants change the code but are slower, with four-case geometric means of $0.961\text{--}0.996\times$.

On both machines, the original tiled matrix-powers kernel remains far below its traffic roof. The CPU evidence points to halo arithmetic and a latency cap that is insensitive to residency. The fixed-volume GPU counters identify tile geometry and access latency more directly. Plane streaming recovers part of the opportunity by changing the halo schedule. To predict speedup, the vertical coordinate still needs a tile-redundancy or access-efficiency term; in its current form, it identifies reusable traffic.

9.6 The conditioning guard on an evolving solver basis

After local tuning, the next gate is numerical. Section 7.3 tested the $u^{-1/2}$ conditioning guard with one start vector. The complete solver provides a stronger diagnostic because its start vector changes at every time step. Table 27 reports the largest width admitted over an integration when four columns are requested. As noted in Section 3.2, the dimension-dependent sufficient condition does not cover the measured production Krylov block. Distributed all-reduce rounding is also not represented by a local condition number alone. The evolving-basis measurements therefore complement the direct orthogonality and referee checks; they do not replace them.

Real Leja points are interpolation nodes selected greedily from a real spectral interval so that each new point maximizes its product of distances from the previously selected points. The Newton recurrence uses these points as shifts to control polynomial growth; the implementation obtains the interval from the Gershgorin enclosure described below [31].

Table 27: Largest Krylov-block width admitted by the implemented guard, and the largest Krylov-block condition number observed over the integration, at a requested width of 4 (Rainbow, idle V100). The guard is $\kappa_2(\mathbf{B}) \leq u^{-1/2} \approx 9.5 \times 10^7$ and controls the fallback; numerical acceptance also uses the orthogonality and referee checks described in the text.

Basis	$n = 31$		$n = 61$	
	Admitted	$\max \kappa_2(\mathbf{B})$	Admitted	$\max \kappa_2(\mathbf{B})$
Monomial	4	6.0×10^6	4	6.0×10^6
Newton (real Leja)	3	8.5×10^7	3	9.3×10^7
Chebyshev	4	4.7×10^7	3	7.9×10^7

At these two grids and widths, the monomial Krylov blocks have the smallest observed condition numbers and are the only ones for which the guard admits all four requested columns. This is a local result, not a general ranking of polynomial bases. Newton and Chebyshev recurrences are designed to control growth at larger degrees, while this experiment tests widths only up to four.

A Gershgorin enclosure is the union of discs centered on each matrix diagonal entry with radius equal to the sum of the absolute off-diagonal entries in that row. It provides a cheap a-priori spectral bound without an eigensolve [17]. Here it admits every requested column up to six, whereas the evolving-basis guard admits at most four, so it is too conservative to select a production width on its own.

The same Krylov blocks were also orthogonalized with TSQR and reorthogonalized Block Gram–Schmidt (the standard algorithm name). On identical $n = 61$, $m = 8$, $s = 4$ Krylov blocks, all three methods produced orthogonality losses between 3.5×10^{-14} and 6.5×10^{-14} , while CholeskyQR2 was fastest in this implementation (Table 28). These measurements compare observed orthogonality and time; they do not establish backward stability for the complete integrator.

Table 28: Orthogonalization on identical Krylov blocks, $n = 61$, Rainbow, idle V100. The table reports observed orthogonality loss and cost; it does not establish full backward stability. Time is basis-plus- $\mathbf{H}$ assembly.

Method	Orthogonality loss	Time	Relative
CholeskyQR2	4.01×10^{-14}	2.248 ms	1.00×
TSQR [32]	3.52×10^{-14}	6.882 ms	3.06×
Block Gram–Schmidt, reorthogonalized	6.47×10^{-14}	2.778 ms	1.24×

9.7 Weak scaling and the largest-common validation stop

With local kernel geometry and numerical width fixed, weak scaling holds local volume near 227,000 rows per GPU. The grid grows from $n = 61$ on one GPU to $n = 77$ on two and $n = 97$ on four. On the final exact-depth arm, two-participant efficiency is 39.7–40.0% for $s = 1$ and 40.7–42.1% for $s = 4$. At four participants, the ranges are 29.9–30.1% and 38.3–38.8%, respectively. All eight distributed points pass validation.

The operator norm grows approximately as n^2 (Section 2). Independent sparse assembly gives 2.19×10^2 at $n = 31$ and 8.74×10^2 at $n = 61$, a ratio of 3.995 compared with $(61/31)^2 = 3.87$. At a fixed step size of $T/100$, inverting the Hochbruck–Lubich bound gives order-of-magnitude Krylov dimensions (Table 29) [9]. The theorem assumes a Hermitian positive semi-definite matrix, or its sign-reversed negative semi-definite case. The pricing operator is non-Hermitian and non-normal, so the bound is used only as a diagnostic scale.

The memory model selects $n = 337$ as the largest odd grid common to every topology, payoff, and width after a 10% reserve; the binding arms are the one-GPU $s = 4$ cases. The predeclared numerical gate then runs Basket at $n = 337$, $s = 4$ without changing the 100 time steps, $m_{\max} = 39$, or tolerance. It converges with maximum $m = 32$, zero unconverged time steps, and maximum residual 9.85×10^{-9} .

Table 29: Diagnostic and observed Krylov dimensions against $m_{\max} = 39$ at a fixed time step of $T/100$. A dash marks the largest-common grid, for which the independent norm diagnostic was not assembled.

n	$\ A\ _1$	Implied m	Observed max. m	Outcome
61	874	15	17	pass
77	1,392	19	20	pass
97	2,209	24	25	pass
337	—	—	32	boundary fail

The separate boundary-state gate requires the three-component polynomial forcing tail to agree with its closed form to the same 10^{-8} tolerance. It fails with an error of 6.40×10^{-5} . The Krylov action indicator can round to zero at $m = 32$. However, it monitors a different quantity from the boundary tail, so tightening the tolerance neither exercises columns 33–39 nor repairs the tail. To test whether poor scaling of the fixed augmentation causes the failure, we reran the pricer with the grid, width, steps, $m_{\max}$, tolerance, basis, orthogonalization, and kernel policy unchanged, and applied the exact power-of-two similarity scaling used by the independent Basket referee. The scaling selects $\eta = 2^{-36}$, reduces $\|B\|_1$ from 4.15×10^{10} to 0.603, and lowers the restored-tail error to 4.41×10^{-13} . The scaled run passes the boundary gate. This diagnostic establishes augmentation scaling as a concrete remediation and does not implicate adaptive Krylov convergence or the shared-memory cap. The original stop remains the predeclared result.

9.8 What the integrator adds to the map

The map identifies two opportunities: expensive reductions can make fewer synchronization rounds valuable, and a large working set can leave traffic for matrix powers to reuse. On one V100, plane streaming realizes part of the vertical opportunity at the width selected by the numerical guard. The promoted dispatch improves the canonical full-solver cases by 1.204–1.297 $\times$ and keeps the full-volume kernel for the small-grid controls. It improves execution at the selected width, but does not establish that a wide Krylov block is always better than $s = 1$.

For a latency-sensitive valuation at $n = 61$, one GPU is enough to hold the problem, and the best four-GPU time is 12–15% slower than the best one-GPU time. Four GPUs become reasonable when a larger grid does not fit on one device or when many independent contracts or parameter scenarios are priced together. A portfolio-style batch can assign separate valuations to otherwise idle devices and prioritize aggregate throughput over the latency of one price. On the measured four-GPU topology, $s = 4$ is 1.78–1.80 $\times$ faster than $s = 1$. Communication avoidance can therefore improve a workload that already requires one distributed valuation. The 0.89 $\times$ same-width result does not justify distributing the $n = 61$ valuation by itself. These conclusions concern semi-discrete solve throughput, not price or Greek accuracy.

10 Conclusion

This thesis studied communication avoidance in a Krylov exponential integrator for two three-asset option PDEs. The coordinates R_v and R_h identify opportunities for cache reuse and fewer reductions. Their unit lines organize the experiments; they are not universal performance crossovers.

The CPU results separate the two mechanisms. Reducing the synchronization count by 11–16 $\times$ gives no wall-clock gain because reductions are not the limiting cost. Cache-blocked matrix powers also remain below their traffic opportunity because halo arithmetic and latency dominate. Direct placement of all eight production grids moves from horizontal-only at $n = 25$, through neither mechanism at $n = 30$ –61, to vertical-only from $n = 74$. The map identifies which mechanism is worth measuring without promising a speedup.

Numerical admissibility is empirical. The $u^{-1/2}$ guard matches 20 of 21 cases, but the available theorem for CholeskyQR2 covers only about 1.25×10^4 at the production dimension. A measured Krylov block reaches 6×10^6 . The numerical claims therefore rely on the guard, successful factorization, measured orthogonality, and agreement with the independent referee over the tested range.

On one V100, plane streaming accepts roughly half the occupancy in exchange for fewer halo instructions and load sectors. The promoted dispatch improves the full solver by a 1.248 $\times$ geometric mean on four selection cases, with a 1.204 $\times$ worst case. Four active held-out cases give a 1.239 $\times$ mean and 1.182 $\times$ worst case, while four small-grid fallback controls remain near parity at 0.991 $\times$.

In the distributed solver, the equal-work $s = 4$ arm is 0.98–1.01 $\times$ the $s = 1$ control on one V100 and 1.78–1.80 $\times$ faster on four V100s. However, same-width strong scaling is only 0.89 $\times$, or about 22% efficiency, and the best four-GPU time is 12–15% slower than the best one-GPU time. The four measured topology rungs do not establish a scaling law. Communication also overlaps device work, so the cost decomposition is not additive. Peer-halo overlap hides 99.7% of copy time but gives no end-to-end improvement within the measured ranges.

The largest-common-grid test stops at its independent boundary gate. At $n = 337$, the Krylov residual passes, but the augmentation-tail error is 6.40×10^{-5} . No validated large-grid price is claimed.

For the stated Basket and Rainbow contracts, both the Krylov and smoothed Hundsdorfer–Verwer ADI solvers show second-order spatial refinement toward independent references and are compared at equal declared accuracy. Basket Delta errors are 0.011–0.020%, and the node-stencil diagonal Gammas are consistent with the references within their 0.05–0.22% uncertainty. Rainbow Delta and Gamma center differences are 0.17–0.24% and 0.54–0.88%, respectively. Off-node Gamma remains limited by extraction. These results apply to the tested three-asset

European contracts and do not compare production pricing systems.

The main conclusion is methodological. Communication-avoiding performance depends on the operator, numerical basis, kernel geometry, topology, and a fair equal-work control. The regime coordinates organize these factors and show where richer models of tile redundancy and distributed synchronization are still needed.

A Supporting studies and future work

The main body concentrates on the regime analysis and the distributed GPU integrator. This appendix retains the supporting temporal benchmark, the construction of the independent financial references, the compact CPU roofline study, and the resulting directions for future work.

A.1 Historical Niesen–Wright time-integrator comparison

Before isolating the CA mechanisms, a historical control checks the temporal integrators and places the Krylov implementation in the Niesen–Wright option-pricing comparison introduced in Section 1. The benchmark plots semi-discrete ODE error against CPU time for five methods, both payoffs, and two grid sizes (Figure 12). Crank–Nicolson and the Douglas–Rachford and Hundsdorfer–Verwer ADI splittings use fixed time-step counts [7, 33, 34]. The other two methods exponentiate the operator. ME uses a scaled, truncated Taylor recurrence with an ∞ -norm early exit [29, 35]. KSM-EI grows its basis until the endpoint defect indicator meets the tolerance.

The referee. The reference is a high-accuracy solution of the same semi-discrete ODE, not an analytic or independent financial price. Before benchmarking each option type, the matrix-exponential referee uses maximum Taylor degree $m = 55$ and $\theta = 9.9$. The internal substep count is

$$s = \left\lceil T \|\tilde{\mathbf{A}}\|_1 / \theta \right\rceil,$$

at tolerance $2^{-53} \approx 1.1 \times 10^{-16}$. The experiment isolates temporal-integration error. It does not include spatial discretization, finite-domain truncation, boundary-condition, or interpolation error, so it cannot establish total price or Greek accuracy. The reported ODE error is the Euclidean norm over the $9 \times 9 \times 9$ grid neighborhood centered on the spot, not the value at one node. This avoids letting one fortuitously accurate node decide the comparison and samples the local surface used to extract price sensitivities.

Role of the benchmark. The Niesen–Wright comparison serves as a historical control on the temporal integrators and their implementation. At $n = 15$ every method reaches the same semi-discrete floor, so further temporal work buys nothing on that grid. The result motivates studying

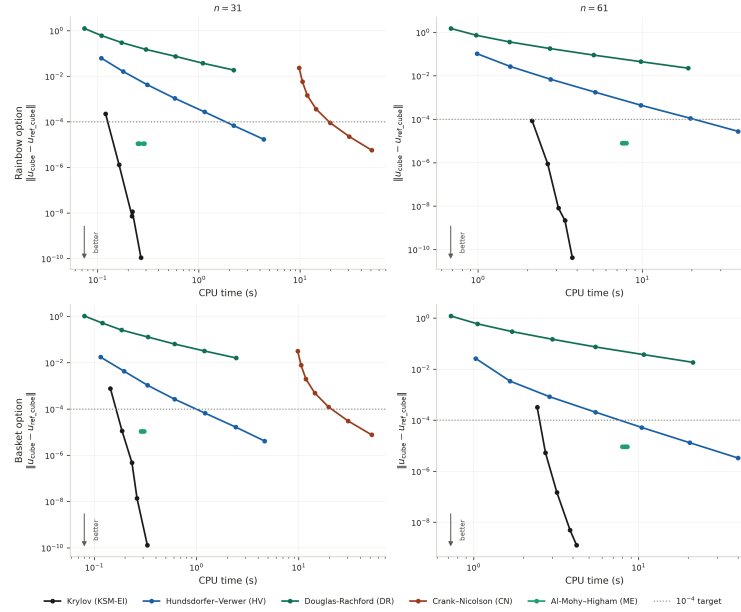


Figure 12: Semi-discrete ODE error vs. CPU time for the five methods, across both payoffs and grid sizes, following the historical Niesen–Wright option-pricing comparison. The Krylov integrator is fastest in the $\sim 10^{-4}$ temporal-accuracy band for these runs; Crank–Nicolson’s non-monotone stiff-mode ringing ($R(-\infty) = -1$) is visible.

the Krylov implementation at larger states but does not establish contemporary pricing competitiveness. That claim would require equal-total-accuracy comparisons with alternatives such as the GPU ADI implementation of Dang, Christara, and Jackson for the same three-asset basket and rainbow contracts [14]. At larger asset counts, simulation methods with dimension reduction provide a more appropriate comparison because the full PDE grid grows exponentially with dimension [20, 21].

A.2 Independent financial references

The preceding benchmark isolates temporal error by comparing each integrator with an accurate solution of the same spatially discretized ODE. This is the right control for time stepping, but it does not test the grid, finite domain, boundary treatment, or interpolation. This section constructs independent financial references, which Section 5 then uses to measure total price error.

The two references have different roles. Neither uses the PDE assembly, boundary closure, grid interpolation, Krylov code, or ADI solvers. Their implementation is a separate translation unit that links no part of the solver library, preventing a reference from inheriting the discretization error of the method it evaluates.

A.2.1 The analytical Rainbow price

For a European call on the minimum of several lognormal assets there is a closed form [36]. Under the risk-neutral measure, $S_i(T) = S_i(0) \exp((r - \sigma_i^2/2)T + \sigma_i \sqrt{T} W_i)$ with $\mathbf{W} \sim \mathcal{N}(0, \mathbf{R})$, and separating the payoff into the exercise indicator and the identity of the minimum asset gives

$$C = \sum_j S_j(0) \Phi_3(d_j^{(1)}, \{d_{ij}\}_{i \neq j}; \mathbf{R}^{(j)}) - K e^{-rT} \Phi_3(d_1^{(2)}, d_2^{(2)}, d_3^{(2)}; \mathbf{R}), \quad (9)$$

where $d_j^{(2)} = (\log(S_j(0)/K) + (r - \sigma_j^2/2)T)/(\sigma_j \sqrt{T})$ and $d_j^{(1)} = d_j^{(2)} + \sigma_j \sqrt{T}$ are the usual Black–Scholes arguments, and, writing $\sigma_{ij}^2 = \sigma_i^2 + \sigma_j^2 - 2\rho_{ij}\sigma_i\sigma_j$ for the pairwise spread volatility,

$$d_{ij} = \frac{\log(S_i(0)/S_j(0)) - \sigma_{ij}^2 T/2}{\sigma_{ij} \sqrt{T}}.$$

The matrix $\mathbf{R}^{(j)}$ is the correlation of the constraint vector—asset j finishes above the strike and below each other asset—after changing to the measure in which asset j is the *numeraire*, the strictly positive traded asset used as the unit in which all other values are expressed. For row order (strike for j , spread $i - j$, spread $k - j$), its diagonal is one and its upper-triangular entries are

$$R_{01}^{(j)} = \frac{\rho_{ij}\sigma_i - \sigma_j}{\sigma_{ij}}, \quad R_{02}^{(j)} = \frac{\rho_{kj}\sigma_k - \sigma_j}{\sigma_{kj}}, \quad R_{12}^{(j)} = \frac{\sigma_i\sigma_k\rho_{ik} - \sigma_i\sigma_j\rho_{ij} - \sigma_k\sigma_j\rho_{kj} + \sigma_j^2}{\sigma_{ij}\sigma_{kj}}.$$

Here $\Phi_3(a, b, c; \mathbf{R})$ is the probability that three correlated standard-normal variables are simultaneously below a , b , and c . Its numerical evaluation is deterministic. The bivariate function Φ_2 uses Genz’s bounded-integral form of Plackett’s identity, which describes how a multivariate-normal probability changes with correlation [37]. For Φ_3 , a Cholesky factor $\mathbf{L}$, satisfying $\mathbf{R} = \mathbf{L}\mathbf{L}^\top$, transforms independent normals into correlated ones and permits conditioning on the first coordinate. Composite Gauss–Legendre quadrature evaluates the remaining one-dimensional integrals as weighted sums at polynomially accurate nodes. No random state is involved.

A.2.2 The randomized quasi-Monte Carlo reference

The Basket has no closed form, so its reference samples the terminal distribution directly. A Sobol sequence is a deterministic low-discrepancy sequence that fills the unit cube more evenly than independent random points. The sampler maps its coordinates through the inverse normal CDF and a correlation Cholesky factor, evaluates the payoff, and discounts it. Points are drawn in powers of two starting at index zero. Truncating or concatenating a Sobol run would lose the balance required by the construction. Before applying the quantile, any coordinate on the closed boundary is moved to the nearest representable interior value.

Uncertainty comes from a Matoušek linear scramble followed by a uniform digital shift [38, 39]. The scramble is an invertible binary transformation, and the shift XORs base-2 digits with independent random bits. Both are bijections of the dyadic digits and preserve uniformity. The estimator averages $R = 16$ independently scrambled replicates and reports a nominal two-sided 95% Student t interval across their means [40]. Sobol points inside one replicate are not treated as independent.

With seed 25,378,095, the adaptive pilot evaluates $2^{14}, \dots, 2^{22}$ points per scramble. Successive-level agreement and the half-width target accept $2^{21} = 2,097,152$. The same replicates select the level and supply the interval, so the reported uncertainty is an empirical randomized-QMC estimate rather than guaranteed fixed-design coverage. The Greek pass reuses the accepted level and seed as common random numbers across spot bumps; it is not a fresh confirmation sample.

A.2.3 Delta and diagonal Gamma

Prices alone do not validate risk sensitivities, so the independent references also provide Greeks, the derivatives of option value with respect to market inputs. Delta measures first-order sensitivity to an asset's spot price, while diagonal Gamma measures curvature with respect to that same spot. They are estimated here by centered differences on the same terminal draws:

$$\Delta_i = \frac{V(S + h_i \mathbf{e}_i) - V(S - h_i \mathbf{e}_i)}{2h_i}, \quad \Gamma_{ii} = \frac{V(S + h_i \mathbf{e}_i) - 2V(S) + V(S - h_i \mathbf{e}_i)}{h_i^2}.$$

The terminal value is proportional to the initial spot, so a bumped path is the center path rescaled. The same Sobol points, scramble, normal draws, and lognormal factors serve all three evaluations exactly. Each derivative is formed inside its scramble before averaging, preserving the covariance from the common draws. A gate verifies that the bump machinery leaves every scramble's center bit for bit unchanged.

Two error terms pull in opposite directions: sampling uncertainty shrinks as the bump widens, whereas the centered difference's $O(h^2)$ truncation error shrinks as it narrows. Adjacent bump sizes estimate the truncation coefficient, and the narrowest bump checks the expected second-order trend. The reported bump minimizes the estimated sum of sampling and truncation errors; the budget keeps the two contributions separate.

A.3 Profile and roofline

With the numerical target and its limiting error established, the discussion now turns to the hardware cost of producing it.

At $n = 61$, the system has 226,981 unknowns, 4.23M nonzeros, and ≈ 49 MB in compressed sparse column form. Over 100 time steps, SpMV and Gram–Schmidt dominate the KSM-EI

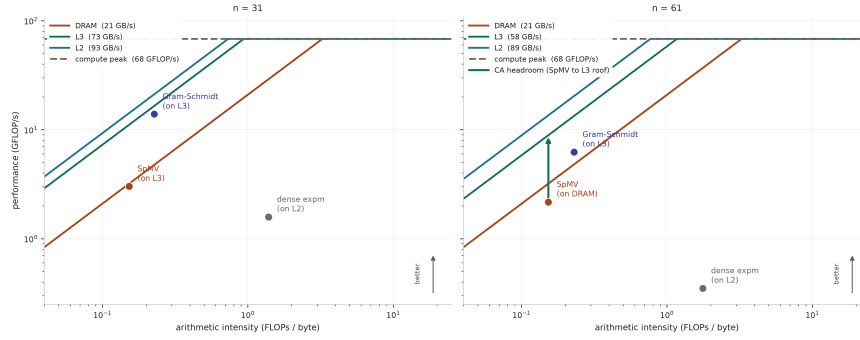


Figure 13: Cache-aware roofline for the two hot kernels at $n = 31$ and $n = 61$. At $n = 61$ the SpMV has spilled to DRAM, the vertical distance from the measured point to the L3 roof at the same arithmetic intensity is exactly the traffic a matrix-powers reformulation would recover.

solve, while the dense $\exp(\mathbf{H}_m)$ is marginal (Table 30). Gram–Schmidt’s share grows with the grid because the Krylov dimension m grows and MGS is quadratic in m . The *other* column contains vector updates, endpoint-defect norm computations, and per-time-step bookkeeping outside the three instrumented kernels.

Table 30: Kernel shares of KSM-EI solve time (basket, tolerance 10^{-8}). $\bar{m}$ is the mean Krylov dimension; *other* is uninstrumented work (vector updates, error estimation, bookkeeping).

Grid	$\bar{m}$	SpMV	Gram–Schmidt	dense expm	other
$n = 31$	7.60	82.9%	11.7%	0.40%	4.9%
$n = 53$	8.42	77.7%	16.8%	0.34%	5.1%
$n = 65$	9.24	76.0%	18.0%	0.21%	5.9%
$n = 73$	9.89	75.2%	18.8%	0.15%	5.9%
$n = 89$	11.34	70.3%	23.8%	0.08%	5.8%

Both kernels remain far below the compute ceiling. The cache-aware roofline explains why and locates the CA opportunity (Figure 13) [41]. Puffin’s measured single-core roofs are 20.9 GB/s for DRAM, 58.2 GB/s for L3, 88.6 GB/s for L2, and 68.1 GFLOP/s for an FMA compute loop.

The attainable gain from moving a kernel one tier up the hierarchy is

$$\text{gain} = \frac{\min(BW_{\text{fast}} \cdot AI, \text{peak})}{\min(BW_{\text{slow}} \cdot AI, \text{peak})}, \quad (10)$$

This expression becomes the bandwidth ratio only while *both* roofs remain sloped. Once the fast roof reaches the compute ceiling, the gain bends downward. A matrix-powers kernel can reach this point because its arithmetic intensity is approximately $s \times$ higher.

A.4 Future work

The next step is to test the regime framework on other operators and hardware. Mixed-precision recurrence with higher-precision Gram accumulation is promising, but it must be evaluated against the dimension-dependent CholeskyQR2 condition, measured orthogonality, and action error [42, 43]. A tile-aware version of R_v could include read redundancy predicted from tile dimensions, stencil radius, and streamed height, using the existing geometry sweeps for validation [22]. H100-class devices, faster links, and more participants would test whether the observed crossover transfers beyond Syngé. Recycling, rational Krylov, and randomized Krylov are relevant lower-communication alternatives [13, 44]. For the pricing application, the immediate numerical extension is a validated higher-order interpolant for off-node Gamma. Mixed Gammas, stress cases, and an optimized GPU ADI reference would follow [14, 15]. Beyond three assets, simulation with dimension reduction becomes more suitable as the tensor grid grows exponentially [20, 21].

References

- [1] Grey Ballard, James Demmel, Olga Holtz, and Oded Schwartz. Minimizing communication in numerical linear algebra. *SIAM Journal on Matrix Analysis and Applications*, 32(3):866–901, 2011.
- [2] Mark Hoemmen. *Communication-Avoiding Krylov Subspace Methods*. PhD thesis, University of California, Berkeley, 2010.
- [3] James Demmel, Mark Hoemmen, Marghoob Mohiyuddin, and Katherine Yelick. Avoiding communication in sparse matrix computations. Technical report, University of California, Berkeley, 2008. IEEE International Symposium on Parallel and Distributed Processing.
- [4] Marghoob Mohiyuddin, Mark Hoemmen, James Demmel, and Katherine Yelick. Minimizing communication in sparse matrix solvers. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis (SC '09)*, 2009.
- [5] Erin Carson. *Communication-Avoiding Krylov Subspace Methods in Theory and Practice*. PhD thesis, University of California, Berkeley, 2015.
- [6] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [7] Willem Hundsdorfer and Jan G. Verwer. *Numerical Solution of Time-Dependent Advection-Diffusion-Reaction Equations*, volume 33 of *Springer Series in Computational Mathematics*. Springer, 2003.
- [8] Yousef Saad. Analysis of some Krylov subspace approximations to the matrix exponential operator. *SIAM Journal on Numerical Analysis*, 29(1):209–228, 1992.
- [9] Marlis Hochbruck and Christian Lubich. On Krylov subspace approximations to the matrix exponential operator. *SIAM Journal on Numerical Analysis*, 34(5):1911–1925, 1997.
- [10] NVIDIA Corporation. CUDA C++ Programming Guide, 2025. URL <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>. NVIDIA developer documentation, as shipped with CUDA Toolkit 12.8.
- [11] Yusaku Yamamoto, Yuji Nakatsukasa, Yuka Yanagisawa, and Takeshi Fukaya. Roundoff error analysis of the CholeskyQR2 algorithm. *Electronic Transactions on Numerical Analysis*, 44:306–326, 2015.
- [12] Mario Berljafa and Stefan Güttel. Parallelization of the rational Arnoldi algorithm. *SIAM Journal on Scientific Computing*, 39(5):S197–S221, 2017. doi: 10.1137/16M1079178.
- [13] Jean-Guillaume de Damas, Laura Grigori, Igor Simunec, and Edouard Timsit. Randomized orthogonalization and Krylov subspace methods: Principles and algorithms, 2025.
- [14] Duy Minh Dang, Christina C. Christara, and Kenneth R. Jackson. A parallel implementation on GPUs of ADI finite difference methods for parabolic PDEs with applications in finance. Technical report, Social Science Research Network, 2010.
- [15] Peter Raahauge. Higher-order finite element solutions of option prices. Finance Working Paper 2004-5, Copenhagen Business School, 2004.
- [16] Advanced Micro Devices, Inc. AOCL performance tuning guide: Introduction, 2025. URL <https://docs.amd.com/r/en-US/63859-AOCL-performance-tuning-guide/Introduction>. Describes NUMA Per Socket on AMD Ryzen, Threadripper, and EPYC systems.

- [17] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.
- [18] Lloyd N. Trefethen and David Bau, III. *Numerical linear algebra*. SIAM, 6 1997.
- [19] C. C. W. Leentvaar and C. W. Oosterlee. On coordinate transformation and grid stretching for sparse grid pricing of basket options. *Journal of Computational and Applied Mathematics*, 222(1): 193–209, 2008. doi: 10.1016/j.cam.2007.10.015.
- [20] Xiaoqun Wang. Dimension reduction techniques in quasi-Monte Carlo methods for option pricing. *INFORMS Journal on Computing*, 21(3):488–504, 2009. doi: 10.1287/ijoc.1080.0304.
- [21] Chengfeng Weng, Xiaoqun Wang, and Zhijian He. Efficient computation of option prices and greeks by quasi-Monte Carlo method with smoothing and dimension reduction. *SIAM Journal on Scientific Computing*, 39(2):B298–B322, 2017. doi: 10.1137/15M1050380.
- [22] Christie L. Alappat, Georg Hager, Olaf Schenk, and Gerhard Wellein. Level-based blocking for sparse matrices: Sparse matrix-power-vector multiplication. *IEEE Transactions on Parallel and Distributed Systems*, 2022. doi: 10.1109/TPDS.2022.3223512.
- [23] Zhaojun Bai, Dan Hu, and Lothar Reichel. A Newton basis GMRES implementation. *IMA Journal of Numerical Analysis*, 14(4):563–581, 1994.
- [24] Anthony T. Chronopoulos and C. William Gear. s -step iterative methods for symmetric linear systems. *Journal of Computational and Applied Mathematics*, 25(2):153–168, 1989.
- [25] Pieter Ghysels and Wim Vanroose. Hiding global synchronization latency in the preconditioned conjugate gradient algorithm. *Parallel Computing*, 40(7):224–238, 2014.
- [26] Dominik Ernst, Georg Hager, Jonas Thies, and Gerhard Wellein. Performance engineering for real and complex tall & skinny matrix multiplication kernels on GPUs. *International Journal of High Performance Computing Applications*, 35(1):5–19, 2021.
- [27] Takeshi Fukaya, Yuji Nakatsukasa, Yuka Yanagisawa, and Yusaku Yamamoto. CholeskyQR2: A simple and communication-avoiding algorithm for computing a tall-skinny QR factorization on a large-scale parallel system. In *Proceedings of the 5th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems (ScalA '14)*, pages 31–38, 2014.
- [28] Ichitaro Yamazaki, Hartwig Anzt, Stanimire Tomov, Mark Hoemmen, and Jack Dongarra. Improving the performance of CA-GMRES on multicores with multiple GPUs. In *Proceedings of the IEEE 28th International Parallel and Distributed Processing Symposium (IPDPS)*, pages 382–391, 2014.
- [29] Awad H. Al-Mohy and Nicholas J. Higham. Computing the action of the matrix exponential, with an application to exponential integrators. *SIAM Journal on Scientific Computing*, 33(2):488–511, 2011.
- [30] NVIDIA Corporation. Nsight Compute Profiling Guide, 2025. URL <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html>. NVIDIA developer documentation, as shipped with CUDA Toolkit 12.8.
- [31] Marco Caliari, Peter Kandolf, Alexander Ostermann, and Stefan Rainer. The Leja method revisited: Backward error analysis for the matrix exponential. *SIAM Journal on Scientific Computing*, 38(3): A1639–A1661, 2016. doi: 10.1137/15M1027620.

- [32] James Demmel, Laura Grigori, Mark Hoemmen, and Julien Langou. Communication-optimal parallel and sequential QR and LU factorizations. *SIAM Journal on Scientific Computing*, 34(1): A206–A239, 2012.
- [33] Jim Douglas and Henry H. Rachford. On the numerical solution of heat conduction problems in two and three space variables. *Transactions of the American Mathematical Society*, 82:421–439, 1956.
- [34] Karel J. in ’t Hout and Sven Foulon. ADI finite difference schemes for option pricing in the Heston model with correlation. *International Journal of Numerical Analysis and Modeling*, 7(2):303–320, 2010.
- [35] Nicholas J. Higham. The scaling and squaring method for the matrix exponential revisited. *SIAM Journal on Matrix Analysis and Applications*, 26(4):1179–1193, 2005.
- [36] Herb Johnson. Options on the maximum or the minimum of several assets. *Journal of Financial and Quantitative Analysis*, 22(3):277–283, 1987. doi: 10.2307/2330963.
- [37] Alan Genz. Numerical computation of multivariate normal probabilities. *Journal of Computational and Graphical Statistics*, 1(2):141–149, 1992. doi: 10.1080/10618600.1992.10477010.
- [38] Jiří Matoušek. On the L2-discrepancy for anchored boxes. *Journal of Complexity*, 14(4):527–556, 1998. doi: 10.1006/jcom.1998.0489.
- [39] Art B. Owen. Scrambled net variance for integrals of smooth functions. *The Annals of Statistics*, 25(4):1541–1562, 1997. doi: 10.1214/aos/1031594731.
- [40] Pierre L’Ecuyer and Christiane Lemieux. Recent advances in randomized quasi-Monte Carlo methods. *Modeling Uncertainty: An Examination of Stochastic Theory, Methods, and Applications*, pages 419–474, 2002. doi: 10.1007/0-306-48102-2_20.
- [41] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [42] Nicholas J. Higham and Theo Mary. Mixed precision algorithms in numerical linear algebra. *Acta Numerica*, 31:347–414, 2022.
- [43] Ahmad Abdelfattah, Hartwig Anzt, Erik G. Boman, Erin Carson, Terry Cojean, Jack Dongarra, Alyson Fox, Mark Gates, Nicholas J. Higham, Xiaoye S. Li, Jennifer Loe, Piotr Luszczek, Srikara Pranesh, Siva Rajamanickam, Tobias Ribizel, Barry F. Smith, Kasia Swirydowicz, Stephen Thomas, Stanimire Tomov, Yaohung M. Tsai, and Ulrike M. Yang. A survey of numerical linear algebra methods utilizing mixed-precision arithmetic. *International Journal of High Performance Computing Applications*, 35(4):344–369, 2021.
- [44] Michael L. Parks, Eric de Sturler, Greg Mackey, Duane D. Johnson, and Spandan Maiti. Recycling Krylov subspaces for sequences of linear systems. *SIAM Journal on Scientific Computing*, 28(5): 1651–1674, 2006.

AI Detailed statement about usage of AI tools

I am aware that I am responsible for all content of this master’s thesis, including the parts where AI tools were used. I am responsible for ensuring that the thesis complies with ethical rules for privacy and publication. To complete this summer project and thesis, I used AI tools as follows:

Name and version	Purpose
OpenAI GPT-5.6 Sol (2026)	Used as a study, editing, and presentation aid. (i) <i>Technical study</i> . Explaining in plainer language the theory I then studied in the primary sources: Krylov approximation of the matrix exponential and its stopping indicator, the s -step communication-avoiding reformulation, the CholeskyQR2 conditioning condition, and roofline and reduction-cost reasoning (Sections 3 and 7 to 9). (ii) <i>Literature and structure</i> . Suggesting connections across numerical linear algebra, performance engineering, and computational finance. Every cited work was read in the original. (iii) <i>Language</i> . Reviewing grammar and clarity and helping me revise the prose against examples of my own academic writing. I checked and accepted each revision. (iv) <i>Scientific presentation</i> . Reviewing figures generated by my plotting scripts in scripts/plots/, including axis labels, panel arrangement, color choice, and captions; no plotted value was changed. Both tools were used interchangeably for these purposes.

Name and version	Purpose
Anthropic Claude Opus 5 (2026)	Used as a study, editing, and presentation aid. (i) <i>Technical study</i> . Explaining in plainer language the theory I then studied in the primary sources: Krylov approximation of the matrix exponential and its stopping indicator, the s -step communication-avoiding reformulation, the CholeskyQR2 conditioning condition, and roofline and reduction-cost reasoning (Sections 3 and 7 to 9). (ii) <i>Literature and structure</i> . Suggesting connections across numerical linear algebra, performance engineering, and computational finance. Every cited work was read in the original. (iii) <i>Language</i> . Reviewing grammar and clarity and helping me revise the prose against examples of my own academic writing and correspondence. I checked and accepted each revision. (iv) <i>Scientific presentation</i> . Reviewing figures generated by my plotting scripts in scripts/plots/, including axis labels, panel arrangement, color choice, and captions; no plotted value was changed. Both tools were used interchangeably for these purposes.

I treated all output as suggestions, checked technical claims against primary literature or my own measurements, and reviewed every edit. The implementation, experiments, measurements, conclusions, and final wording are my own work.